College of Engineering, Environment, and Computing
School of Science

MSc Data Science
7150CEM Data Science Project

Project Report

Material-Agnostic Feature Fusion for Tool Condition Monitoring: Bridging the Generalization Gap Between Steel and Titanium Alloys

By

Manoj Kumar Gunupudi
SID: 15917205

Supervisor: Ammar Al-Bazi

Submitted in partial fulfilment of the requirements for the Degree of Master of Science in Data Science

Academic Year: 2025/26

Declaration of Originality

I declare that this project is all my own work and has not been copied in part or in whole from any other source except where duly acknowledged. As such, all use of previously published work (from books, journals, magazines, internet etc.) has been acknowledged by citation within the main report to an item in the References or Bibliography lists. I also agree that an electronic copy of this project may be stored and used for the purposes of plagiarism prevention and detection.

Statement of copyright

I acknowledge that the copyright of this project report, and any product developed as part of the project, belong to Coventry University. Support, including funding, is available to commercialize products and services developed by staff and students. Any revenue that is generated is split with the inventor/s of the product or service. For further information please see www.coventry.ac.uk/ipr or contact ipr@coventry.ac.uk.

Statement of ethical engagement

I declare that a proposal for this project has been submitted to the Coventry University ethics monitoring website (https://ethics.coventry.ac.uk/) and that the application number is listed below (Note: Projects without an ethical application number will be rejected for marking).

Signed: Gunupudi Manoj Kumar

Date: 22/02/2026

Please complete all fields.

First Name	**Manoj Kumar**
Last Name	**Gunupudi**
Student ID Number	**15917205**
Ethics Application Number	**P192917**
1st Supervisor Name	
2nd Supervisor Name	

Abstract

Tool condition monitoring (TCM) is an important practice in the contemporary manufacturing processes since it helps in predictive maintenance and enhances the reliability of the operations. Nonetheless, the majority of the available machine learning-based TCM models are trained and tested on single-material datasets, which restricts their power to generalise the models in various machining settings. The work deals with the issue of the generalisation between materials by suggesting that prediction of tool wear can be performed using a material-agnostic feature fusion model.

The study examines how a deep learning model that was trained using steel machining data will perform when used in predicting titanium machining environments. A baseline hybrid convolutional neural network and long short-term memory (CNN-LSTM) model is built, and then the Material-Agnostic Feature Fusion (MAFF) model with the use of domain-invariant fusion mechanism is introduced. These models are tested with a well-defined framework which entails zero-shot cross-domain testing, signal ablation analysis, latency analysis, and few-shot adaptation.

The experimental evidence proves that the two models severely deteriorate their performance when transferred to zero-shot transfer, which validates the fact that there is a generalisation gap between steel and titanium machining. Nevertheless, the suggested MAFF model becomes significantly better when there is a few-shot adaptation, where predictive accuracy recovers faster with little target-domain data. Moreover, the model meets real-time deployment requirements that have low inference latency.

The results point to the fact that, material-agnostic zero-shot prediction is a difficult task, and feature fusion can be used to increase the flexibility in situations with limited data. This paper brings about more intelligent manufacturing systems that are stronger and can be scaled.

Keywords

Condition Monitoring of tools, machine learning, feature fusion, domain generalisation, CNN-LSTM, predictive maintenance, cross-material learning, smart manufacturing.

Table of Contents

Abstract 3
1. Introduction 7
1.1 Background of the Study 7
1.2 Problem Statement 8
1.3 aim of the study 8
1.4 Study Objectives 8
1.5 Research Question 8
1.6 Importance of the Study 9
1.7 Scope of the Study 9
1.8 Outline of the Dissertation 9
2. Literature Review 10
2.1 Introduction 10
2.2 Machine Learning in Monitoring in Machining and Manufacturing 10
2.3 Tool Condition Monitoring Systems and Predictive Maintenance 13
2.4 Sensor Fusion and Multi-Modal Monitoring in Manufacturing 15
2.5 Research Gap 17
3. Research Methodology 18
3.1 Introduction 18
3.2 Data Set Selection and Justification 18
3.3 Research Philosophy and Design 19
3.4 Channel Selection Rationale 21
3.5 Preprocessing and Windowing 21
3.6 Experimental Evaluation Framework 21
3.7 Few-Shot Adaptation and Statistical Validation 21
4. Technical Aspects / Solution Design 22
4.1 Introduction 22
4.2 Computational Environment and Libraries 22
4.3 Signal Representation and Input Format 22
4.4 CNNLSTMBaseline Architecture 23
4.4.1 Convolutional Feature Extraction 23
4.4.2 LSTM Temporal Encoding and Regression Head 23
4.5 Material-Agnostic Feature Fusion Model 23
4.5.1 Channel Attention Mechanism 23

4.5.2 Bottleneck Residual Block 24
4.6 Training Configuration 24
4.7 Evaluation Metrics 24
5. Implementation 25
5.1 Introduction 25
5.2 Environment Configuration 25
5.3 Data Loading 26
5.4 Exploratory Data Analysis 28
5.5 Preprocessing Pipeline 28
5.6 Dataset and DataLoader Construction 29
5.7 Model Architecture Implementation 30
5.8 Training Loop Implementation 31
5.9 Evaluation, Ablation, and Latency Implementation 32
5.10 Few-Shot Adaptation and Statistical Validation 34
5.11 Chapter Summary 35
6. Testing 36
6.1 Introduction 36
6.2 Tier 1: Source-Domain Test Execution 38
6.3 Tier 1: Zero-Shot Cross-Domain Transfer 38
6.4 Tier 2: Signal Ablation Execution 38
6.5 Tier 3: Latency Verification 38
6.6 Few-Shot Adaptation Execution 39
6.7 Statistical Validation Execution 39
7. Results and Evaluation 40
7.1 Introduction 40
7.2 Source-Domain Performance (Tier 1 In-Domain) 40
7.3 Zero-Shot Cross-Domain Transfer (Tier 1 Cross-Domain) 41
7.4 Signal Ablation Results (Tier 2) 42
7.5 Latency Results (Tier 3) 42
7.6 Few-Shot Adaptation Results 43
7.7 Statistical Validation 44
7.8 Summary of Findings 45
8. Discussion and Reflection 46
8.1 Introduction 46
8.2 Interpretation of Major Findings 46

8.3 Research Contribution 46
8.4 Limitations 46
8.5 Future Work 46
8.6 Ethical Reflection 46
8.7 Critical Analysis of Model Limitations 46
9. Project Management 47
9.1 Overview 47
9.2 Project Timeline and Planning 47
9.3 Meetings and Integration of Supervisory Feedback 47
9.4 Risk Management 47
9.5 project execution and monitoring 48
10. Conclusion 49
10.1 Introduction 49
10.2 Summary of Research 49
10.3 Key Findings 49
10.4 Answer to Research Question 50
10.5 Contributions of Study 50
10.6 Final Remarks 50
References 52
APPENDIX A- Source Code Snippets 56
APPENDIX B- GitHub Repository Link 62
Appendix C-LOGBOOK TABLE 63
Appendix D – Ethics Approval 64

1. Introduction

1.1 Background of the Study

The study has a background which will be discussed as follows.

The transformation of smart manufacturing has greatly enhanced efficiency of industries, automation, and reliability. The combination of artificial intelligence (AI) and machine learning (ML) allows making decisions based on data, predictive maintenance, and real-time monitoring, which contributes to optimisation of processes and high-performing systems (Abraham and Gogotsi, 2026; Kollu and Kittur, 2025).

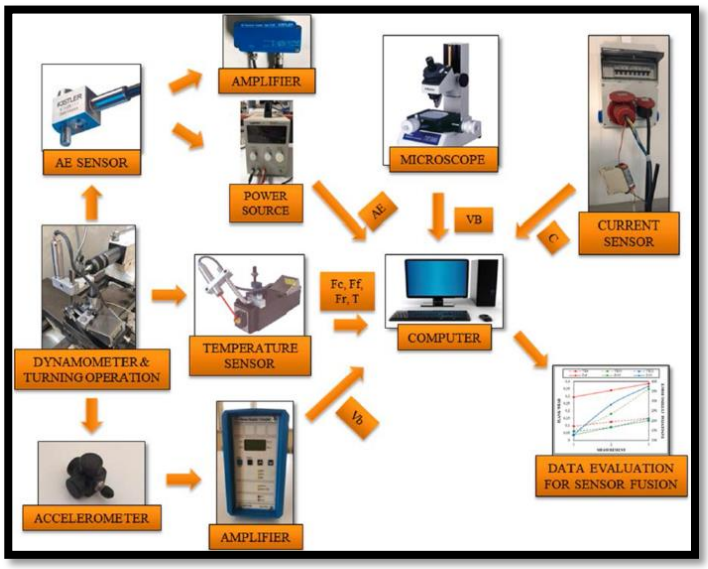

Figure 1.1 - Artificial Intelligence for Tool Condition Monitoring (https://link.springer.com/article/10.1007/s10845-022-01923-2)

Tool Condition Monitoring (TCM) plays a significant role in machining processes, because the wear on tools directly influences the quality of products, precision and efficiency of operations. Conventional methods like manual inspection, threshold-based methods, etc. have limitations in their ability to identify the complexities between sensor signals and wear behaviour. Conversely, ML-based methods use multi-sensory information, including vibration, cutting force and acoustic emissions, to enhance the accuracy of fault detection and wear prediction (Kaur, Kumar and Aggarwal, 2025).

The latest trends in digital and intelligent manufacturing only contribute to the significance of smart monitoring systems, as AI-based solutions allow optimising in real-time and identifying defects (Liu et al., 2025; Kishor et al., 2026).

1.2 Problem Statement

Although there has been progress in the ML-based TCM, a significant drawback is the lack of generalisation among various machining conditions. Majority of the models have been trained and tested on one dataset and cannot work on other materials.

This problem can be seen in cross-material cases, like the transfer of models to titanium machining. The variations of thermo-mechanical characteristics produce a significant change in cutting forces, heat generation and wear patterns, which cause domain shift and impaired model performance.

Moreover, current methods need considerable labelled data on each material, making this unrealistic in industrial settings based on the high cost of data collection. Consequently, models which can make generalisation through materials with minimal information of the target-domain are needed.

1.3 aim of the study

The purpose of this work is to create a material-agnostic feature fusion model of tool condition monitoring, which will minimise the generalisation gap between machining materials of different types, especially between steel and titanium.

1.4 Study Objectives

This study aims to:

1. To analyse limitations of existing ML-based TCM approaches in cross-material scenarios.
2. To create a baseline CNN-LSTM tool wear prediction model.
3. To create a domain-invariant feature fusion model (MAFF).
4. To compare the performance of zero-shot and few-shot transfer.
5. To analyze signal ablation in sensor modalities.
6. To determine the real-time deployment viability by evaluating the latency.

1.5 Research Question

Will a domain-invariant feature fusion model work reliably in predicting tool wear when transferred between steel machining and titanium machining without retraining?

1.6 Importance of the Study

This work is scholarly because it helps to learn more about the concept of cross-domain generalisation in ML-based TCM, namely, material variability.

In terms of industry, material-agnostic models have the ability to decrease reliance on large labeled datasets and enhance scalability in a wide variety of machining settings. This is in line with Industry 4.0 trends, which allow predictive maintenance to be efficient, minimize downtime and enhance decision-making.

1.7 Scope of the Study

This study aims at determining ML-based tool condition monitoring based on multi-sensor machining data and cross-material generalisation of steel and titanium.

The experiment involves deep learning models, feature fusion methods, and testing in a zero-shot and few-shot setup with regression indicators like R2 and MAPE.

Nonetheless, they do not include real-time industrial implementation and hardware connectivity, as the experiments are run on offline data sets.

1.8 Outline of the Dissertation

The dissertation will be structured in the following way:

- Chapter 2: Literature review on TCM, ML, and domain generalisation
- Chapter 3: Research methodology and experimental design
- Chapter 4: System architecture and technical design
- Chapter 5: Implementation details
- Chapter 6: Testing and validation
- Chapter 7: Results and evaluation
- Chapter 8: Discussion and limitations
- Chapter 9: Project Management
- Chapter 10: Conclusion and contributions

2. Literature Review

2.1 Introduction

Smart manufacturing has redefined the industrial systems by incorporating artificial intelligence (AI), machine learning (ML), and data-driven decision-making. These technologies can be used to have predictive maintenance, real-time monitoring and better operational efficiency. Tool Condition Monitoring (TCM) is specifically important in high-precision industries because tool wear directly affects the quality of the product, its cost, as well as the reliability of the machine.

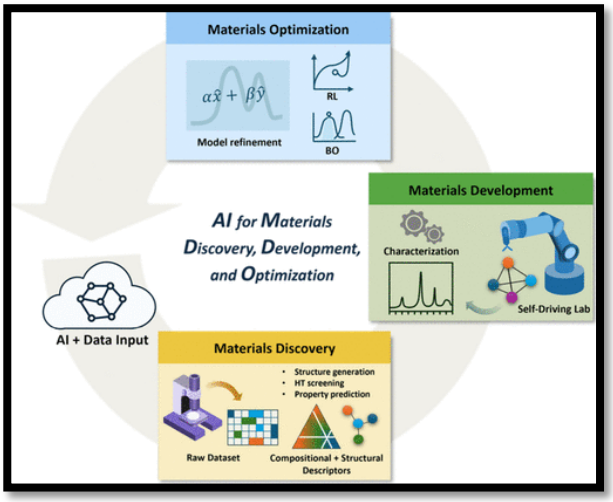

Figure 2.1 - Artificial Intelligence for Material Engineering (Madika et al., 2025)

Multi-sensor data is analysed by the ML-based monitoring systems to identify wear patterns and predict failures more efficiently than the conventional rule-based models (Kaur et al., 2025). Nevertheless, one major weakness is that most of the models are developed using special materials or conditions and cannot be generalised to other machining conditions. The chapter is a literature review of four topics, namely ML in manufacturing (2.2), TCM and predictive maintenance (2.3), sensor fusion (2.4), and domain generalisation (2.5), which results in the research gap.

2.2 Machine Learning in Monitoring in Machining and Manufacturing

The use of machine learning in machining has been predominantly applied in tool wear prediction, process optimization, and fault detection. Conventional models like SVM and Random Forest, and deep learning models like CNNs, LSTMs, and Transformers have proven to be effective in interpreting complex sensor data (Kaur et al., 2025; Rane, 2023).

Research indicates that ML increases machining efficiency and quality (Cevik and Sariisik, 2025), whereas physics-informed models increase the material behaviour representation (Bobbili et al.,

2025). Digital twins and hybrid AI systems also enhance the accuracy of monitoring and real-time analysis (Saatci et al., 2025; Stephen, 2025).

Irrespective of these developments, the majority of models are confined to particular datasets or materials and are unable to generalise across materials (Kollu and Kittur, 2025).

Author(s) & Year	**Focus Area**	**Method**	**Key Findings**	**Limitations**
Kaur et al. (2025)	AI in machining operations	Systematic review of ML, DL, AI in machining	AI is now widely used to optimise processes, predict tool wear, and detect faults	Generalisation across materials not addressed
Cevik and Sariisik (2025)	CNC milling parameter optimisation	ML-driven cutting parameter selection	ML improves surface finish quality and reduces cutting defects	Specific to milling; not tested on other machining types
Bobbili et al. (2025)	Physics-informed neural networks in manufacturing	Physics-inspired ML for material response	Physics-guided models better represent complex material behaviour in manufacturing	High complexity; limited to specific materials
Naeim et al. (2025)	Fatigue life prediction for steel	ML with mechanical properties and stress state data	ML can predict fatigue performance in steel with high accuracy	Focuses only on steel; limited to structural materials
Omar et al. (2023)	Structural health monitoring and crack growth	ML on dynamic response data	ML tracks complex engineering structures and predicts degradation effectively	Application limited to structural components, not cutting tools

Rane (2023)	Transformer models in materials science	Attention mechanisms and transformer architectures	Transformers learn long-range dependencies in time-series industrial signals	Limited real-world industrial validation
Hassan et al. (2025)	Physics-guided attention for defect detection	Physics-guided deep learning	Physics-guided attention improves generalisation in defect detection models	Tested on limited material types
Kollu and Kittur (2025)	Deep learning in mechanical engineering	Systematic review of DL applications	Deep learning is widely applied across mechanical engineering domains	Lacks focus on cross-material generalisation challenges
Saatci et al. (2025)	Hybrid AI for tool wear monitoring	Hybrid ML system combining multiple algorithms	Combining ML techniques significantly enhances monitoring accuracy	Evaluated only on specific factory datasets
Stephen (2025)	Digital twin and ML for drilling monitoring	Real-time digital twin with ML	Digital twins combined with ML provide effective real-time tool monitoring	Focused on composite materials in robotic drilling only

Table 2.1: Review Matrix – Machine Learning Applications in Machining and Manufacturing Monitoring

Majority of ML/DL machining models only work within the same dataset/material and seldom gives a hint that it can rely when switching between steel and titanium. There is little evidence to support the idea that there is no material-specific retraining deployment, and actual cross-material portability is unknown. Multi-sensor fusion and physics-guided/digital twin concepts are not commonly tested in-domain, but when there is a very strong domain shift and also when one uses thermal-loads that are different. The state of uncertainty and failure-mode reporting is poor, thus cross-material tool-wear prediction which can be trusted to be a gap (e.g., R 2) is still poor (e.g. R 2>0.85).

2.3 Tool Condition Monitoring Systems and Predictive Maintenance

TCM systems allow reactive to be replaced with predictive maintenance by analysing sensor data to predict the health and useful life of tools. Multi-sensor ML systems enhance the accuracy and reliability of the monitoring significantly (Saatci et al., 2025; Nazir and Shao, 2021).

The new technologies like digital twins and adaptive monitoring systems offer real-time information and optimisation of processes (Zhang et al., 2025; Stephen, 2025). Adaptability to different conditions is further improved by meta-learning and adversarial methods (Meng et al., 2024; Shao et al., 2025).

Nevertheless, these processes are very reliant on certain processes or materials and cannot be applied through various machining settings.

Author(s) & Year	Focus Area	Method	Key Findings	Limitations
Saatci et al. (2025)	Hybrid AI for tool wear monitoring	Hybrid ML combining multiple techniques	Hybrid systems enhance monitoring accuracy and predictive reliability	Limited to specific factory or machining conditions
Nazir and Shao (2021)	Online TCM for ultrasonic metal welding	Sensor fusion with ML	Multi-sensor fusion enables accurate real-time tool condition assessment	Method specific to ultrasonic metal welding only
Stephen (2025)	Real-time tool monitoring in robotic drilling	Digital twin combined with ML	Digital twin and ML together enable real-time quality and condition monitoring	Focused on composite materials; limited generalisation
Zhang et al. (2025)	Multi-sensor fusion for machine condition monitoring	Self-adaptive ML with multi-sensor inputs	Multi-sensor fusion with adaptive models achieves reliable real-time monitoring	High system complexity; requires many sensors

Meng et al. (2024)	TCM across different welding activities	Meta-learning-based domain generalisation	Meta-learning framework improves tool condition monitoring across different welding tasks	Domain coverage limited to welding operations
Shao et al. (2025)	Tool wear monitoring on unseen machines	Adversarial domain generalisation	Adversarial methods learn features that transfer across different machines and setups	Not yet validated across wide range of machining materials
Liu et al. (2025)	Deep learning for DED additive manufacturing monitoring	Deep learning techniques review	Deep learning effectively monitors and controls direct energy deposition processes	Review only; no new system proposed
Kishor et al. (2026)	In-situ monitoring in laser additive manufacturing	ML for in-situ process monitoring	ML can predict residual stress and detect defects during laser additive fabrication	Specific to laser-based additive processes
Łach and Svyetlichnyy (2026)	AI-augmented digital twins for powder bed fusion	Physics-based simulation combined with AI digital twin	AI digital twins enable smart monitoring and adaptive control in manufacturing	Focused on powder bed fusion; not directly on TCM
Zhu et al. (2021)	ML-based condition monitoring in additive manufacturing	Review of ML approaches in additive manufacturing	ML is widely used for condition monitoring in metal additive manufacturing	Methods are highly specific to additive processes

Table 2.2: Review Matrix – Tool Condition Monitoring Systems and Predictive Maintenance

The vast majority of TCM systems (Multi-sensor ML, digital twins, adaptive monitoring) have been demonstrated in just a single process or a single factory and thus do not scale to other machining environments. Although accuracies can be good, they normally apply to a given material or operation (welding/drilling/additive), rather than an overall machining across mediums. The meta-learning and adversarial approaches are potentially interesting, yet they are not tested on a broad selection of machining materials and regimes. Multi- sensor fusion makes it even more complex and yet does not have solid evidence of portability in practical uses. Then the key disconnect is an effective TCM method working on various materials/ environments without retraining process or material specific.

2.4 Sensor Fusion and Multi-Modal Monitoring in Manufacturing

Sensor fusion integrates data from multiple sources (e.g., vibration, acoustic, temperature) to improve monitoring accuracy. Studies demonstrate that multi-sensor systems outperform single-sensor approaches in detecting complex failure patterns (Prodhan et al., 2023; Wang et al., 2026).

Advanced techniques such as latent representation learning and AI-driven fusion models enable better alignment of heterogeneous sensor data (Fan, n.d.; Zhang et al., 2025). Applications in additive manufacturing also highlight improved quality control through multi-modal monitoring (Papatheodorou et al., 2025).

Despite these improvements, current sensor fusion approaches are not designed for cross-material generalisation and often fail when applied to different materials such as steel and titanium.

Author(s) & Year	Focus Area	Method	Key Findings
Prodhan et al. (2023)	AI-powered sensor fusion for failure prediction	Real-time sensor fusion with AI	Sensor fusion with ML enables accurate identification of complex failure patterns
Wang et al. (2026)	Dual-modal defect monitoring in additive manufacturing	Infrared and visible light sensor integration	Multi-modal sensing improves anomaly detection accuracy in fabrication processes
Papatheodorou et al. (2025)	Sensor fusion in laser powder bed fusion	Multi-sensor monitoring and control	Multi-sensor fusion systems improve monitoring

			reliability in high-precision manufacturing
Fan (n.d.)	Latent representation learning for sensor fusion	Probabilistic structural health monitoring	Latent feature representations align different sensor sources effectively
Nazir and Shao (2021)	Online sensor fusion for tool monitoring	Multi-sensor fusion with ML for welding	Combining multiple sensor signals improves tool condition monitoring accuracy
Zhang et al. (2025)	Smart tool condition monitoring via sensor fusion	Self-adaptive multi-sensor fusion with digital twin	Adaptive multi-sensor systems enable real-time monitoring with high precision
Kuşoglu et al. (2025)	Multi-process monitoring in laser powder bed fusion	Large-scale interlaboratory study	Unified monitoring frameworks that span the whole process chain improve consistency
Ye et al. (2025)	Quality control in laser directed energy deposition	Online multi-sensor quality monitoring review	Multiple sensors working together provide effective online quality control
Gao et al. (2025)	GAN-based microstructural mapping from surface profiles	Generative adversarial network for microstructure prediction	GANs can predict internal microstructure from surface profiles using sensor data
Papatheodorou et al. (2025)	Sensor fusion strategies in laser manufacturing	Review of fusion monitoring strategies	Recent advances show sensor fusion is essential for real-time quality assurance

Table 2.3: Review Matrix – Sensor Fusion and Multi-Modal Monitoring

Most sensor-fusion investigations demonstrate enhanced accuracy with single sensor, although these are typically characterized by measurements of only one material, or only one controlled process, and real cross-material portability is unfamiliar. As a result, fusion models tend to memorize signal patterns applied to materials, i.e. they are prone to breaking (when changing) between steel and titanium where the heat and cutting mechanism vary dramatically. Very little work has specifically designed fusion on domain shift (cross-material generalisation) and not merely in-domain accuracy. Assessments in the vein of benchmarks (train on steel, test on titanium) and transparent failure/uncertainty reporting are uncommon. The gap is thus a union framework, which remains stable with a variety of materials without the retraining or re-tuning of the material required.

2.5 Research Gap

The literature shows that there has been immense advancement in ML-based monitoring, sensor fusion and domain adaptation. Nevertheless, there are main shortcomings. Majority of TCM models are conditioned with single material datasets and do not generalise to other machining conditions.

The current domain generalisation strategies are also limited to particular processes, and have not been tested on fundamentally different materials like steel and titanium. Likewise, sensor fusion can enhance accuracy and learn material-invariant features but not material-invariant.

Thus, the research gap to be filled is the creation of material-agnostic feature fusion that can recognize the consistency of wear patterns of various materials. Filling this void would contribute greatly to the scalability and feasibility of smart manufacturing intelligent monitoring systems.

3. Research Methodology

3.1 Introduction

This chapter gives the methodology of the study of material-agnostic feature fusion as a tool condition monitoring method. The critical problem solved is the generalisation of machine learning models in other machining materials because of the different physical properties. The research design is an experiment that involves cross-domain analysis (zero-shot), signal ablation, and latency analysis. An experiment on few-shot adaptation is also performed to test performance recovery with a small amount of target-domain data.

3.2 Data Set Selection and Justification

Two datasets are used to evaluate cross-material generalisation. The dataset used is milling data of both steel and cast iron at different conditions of machining, which allows the training of strong models. The target dataset is machining of titanium alloy, and it offers various wear properties, and thus, it can be used in cross-domain testing.

A very strict zero-shot environment is employed where no target data is involved in training. Only a small set is allocated to few-shot adaptation, with the rest being used to test.

Table 3.1: Dataset Summary

Feature	Source Dataset	Target Dataset
Material	Steel, Cast Iron	Titanium Alloy
Process	Milling	Milling
Sensors	Vibration, AE, Current	Same as source
Usage	Training & Testing	Testing (Zero-shot)
Label	Flank Wear	Flank Wear

To maintain similarity, the common sensor channels are used in both datasets. Standard preprocessing pipeline is used, which consists of signal segmentation and per-window normalisation.

Table 3.2: Data Processing Steps

Step	Description
Segmentation	Signals divided into fixed windows
Normalisation	Z-score applied per window
Label Assignment	Wear values mapped to each window
Feature Input	Raw time-series signals used

This approach ensures fair comparison between domains and supports reliable evaluation of cross-material generalisation.

3.3 Research Philosophy and Design

The research adheres to the positivist philosophy, where tool wear is considered to be an objective and quantifiable variable. It utilises a deductive methodology, in which the current machine learning and domain generalisation concepts are put to the test.

The approach is founded on the Design Science Research (DSR) framework, which comprises of design, construction, and evaluation stages. CNN-LSTM baseline and MAFF model are trained and designed based on source data and then assessed through the use of the zero-shot experiments, few-shot experiments, and latency experiments.

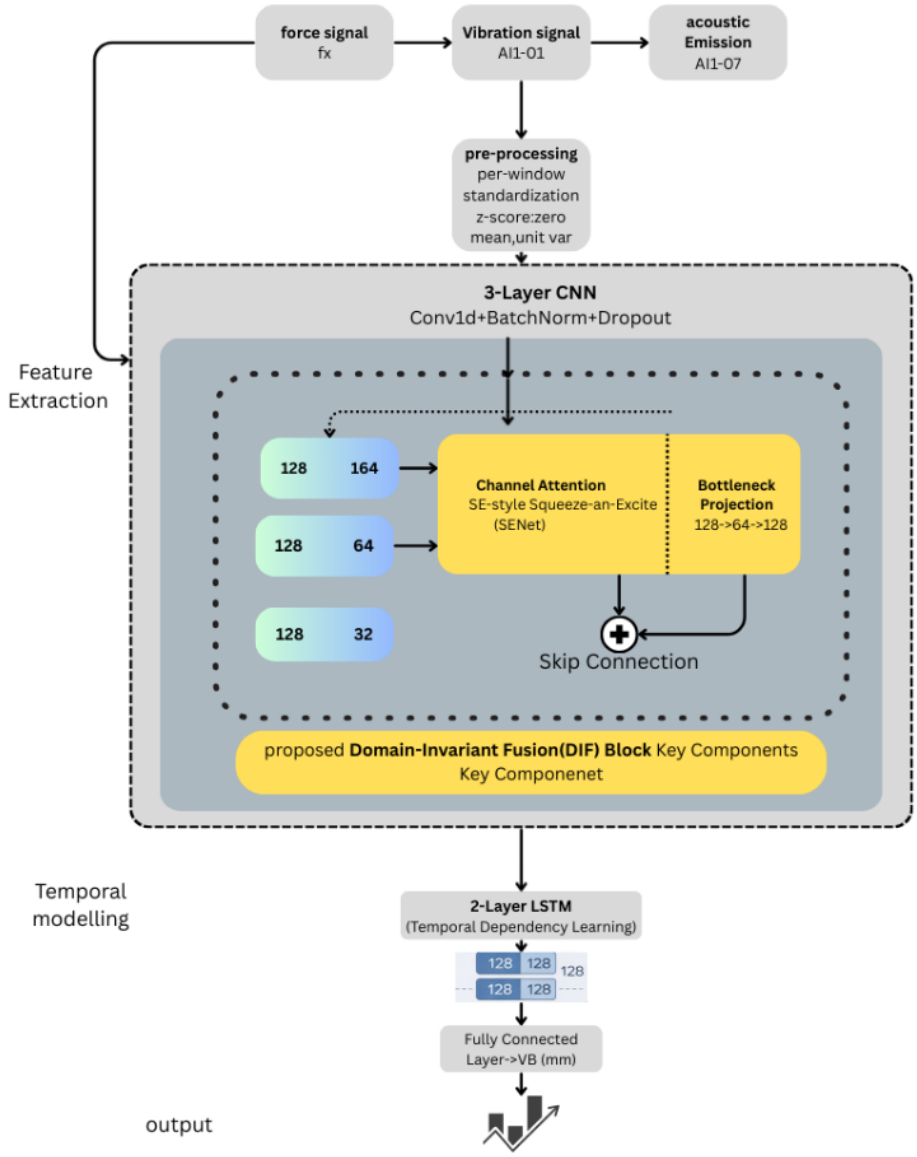

Fig 3.1: Proposed Architectural Workflow of MAFF Model

Multi-sensor inputs (force, vibration, acoustic emission) are used in the system and these are initially normalised with z-score. A 3-layer CNN is used to extract the spatial features, and a Domain-Invariant Fusion (DIF) block is used, which performs channel attention and feature refinement. The extracted

features are then inputted into a 2-layer LSTM in order to learn temporal patterns. Lastly, tool wear (VB in mm) is predicted using a regression layer.

This design can be used to achieve successful multi-sensor learning and cross-domain generalisation.

3.3.1 Research Onion Framework

The research design of this study is further organized with the aid of the research onion framework which gives a systematic approach to the definition of research methodology with the help of various layers such as philosophy, approach, strategy and methods.

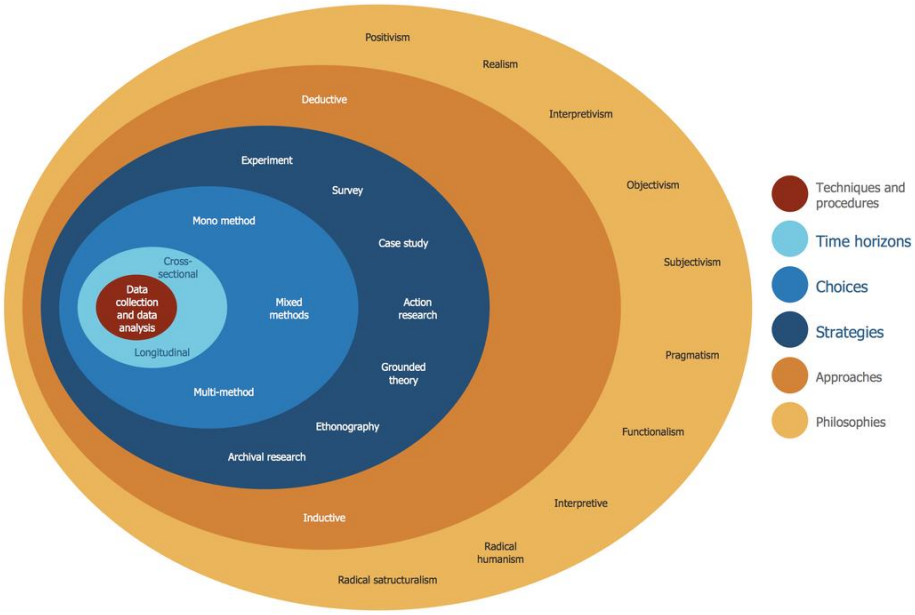

Fig 3.2: Onion Framework

At the surface, the study is a positivist philosophy because the research is founded on the objective measurement of the tool wear and quantitative analysis of the model performance in terms of such statistical measures like R 2, MAE and MAPE. Emphasis is placed on objective verification as opposed to subjective understanding.

Regarding research method, the research takes a deductive research approach, in which available hypotheses in the fields of machine learning, domain generalisation and tool condition monitoring are adopted to develop hypotheses about cross-material transferability. These hypotheses are then tested through experimental evaluation.

This structured use of the research onion framework ensures methodological clarity and aligns the research design with established academic standards.

3.4 Channel Selection Rationale

The reason is that the channel must be selected according to the products and services offered by the company.

Before the actual training, the channel selection is conducted by a linear baseline based on cross-validated performance of regression. The three channel combination selected is spindle motor current, work piece table vibration and spindle acoustic emission signal, which are the three physical phenomena that are complementary to each other in the tool-workpiece contact region. The channels have physical counterparts in the target domain, allowing transfer of materials at cross material without channel level adjustment.

3.5 Preprocessing and Windowing

A consistent pipeline is applied to both datasets. Raw signals are divided into non-overlapping windows with fixed length, each one of which retains its flank wear label and record identifiers. Normalisation of per-window z-score is done on a window basis and not on an over dataset basis, eliminating absolute amplitude differences between the two datasets but maintaining the relative shape of the temporal signal required in cross-material transfer.

3.6 Experimental Evaluation Framework

Tier 1 evaluates cross-domain reliability in two scenarios, one of a held-out in-domain test of unseen source-domain cases, and the other of zero-shot performance on the entire target domain without parameter updates. A pre-specified MAPE degradation value is used to ascertain the presence of a significant shift in domain. Tier 2 Tier 2 applies counterfactual signal ablation to zero individual channels of the target-domain input to determine which modalities the model is dependent on. The Tier 3 checks the latency of inference that occurs on a single window to ensure that the deployment requirement of less than 50 ms per window is met in real-time.

3.7 Few-Shot Adaptation and Statistical Validation

Few-shot experiment Fine-tunes each model on early-wear target-domain cycles, fixing the convolutional backbone and updating only the temporal encoding and regression components. The statistical validation is done by a Wilcoxon signed-rank test to compare the errors between models on the held-out source-domain test set, and bootstrap confidence interval to measure the uncertainty in the R2 without distributional assumptions.

4. Technical Aspects / Solution Design

4.1 Introduction

This chapter presents the technical design and implementation of the deep learning models developed based on the methodology outlined in Chapter 3. While Chapter 3 defined the datasets, preprocessing steps, and experimental framework, this chapter focuses on the architectural design, system configuration, and modelling approach used to implement the proposed solution.

It is important to note that this chapter does not present experimental results. The detailed performance evaluation, results analysis, and discussion are presented in Chapter 7. This chapter is limited to the design and justification of the models and their components.

The chapter includes the computational environment, input signal representation, detailed architecture of both the baseline and proposed models, training configuration, and evaluation metrics. Both models share a common convolutional backbone to ensure that differences in performance are attributed to the domain-invariant fusion mechanism rather than feature extraction differences.

4.2 Computational Environment and Libraries

The pipeline was implemented in Python and run in a Python GPU enabled notebook setup in Kaggle. The deep learning framework that is offered in PyTorch is used due to the dynamic computational graph that facilitates the creation of custom attention and residual modules. Numpy is used to perform array operations throughout the preprocessing pipeline and scikit-learn is used to perform the Ridge regression baseline used to conduct channel selection and computation of evaluation metrics. SciPy provides Wilcoxon signed-rank test which is used in statistical validation. Kaggle environment supports reproducible and containerised implementation with acceleration through the CUDA architecture eliminating hardware-dependence on result replication.

4.3 Signal Representation and Input Format

Following the preprocessing and channel selection strategy defined in Chapter 3, the model inputs consist of. The model inputs consist of three-channel fixed length time-series windows obtained as a result of the chosen sensor data (spindle motor DC current, smcDC, workpiece table vibration, vibtable and spindle acoustic emission, AEspindle). These three channels are a combination of complementary mechanical, vibratory and acoustic phenomena at the tool-workpiece interface. The length of the window is maintained at the same length in both datasets so as to enable direct transfer of models without alteration of the architecture. All the windows are z-score normalised individually and then fed into the model to get the two datasets on the same amplitude scale. Based on the experimental design described in Chapter 3, two models are developed for comparison.

4.4 CNNLSTMBaseline Architecture

The CNNLSTMBaseline is used as the domain-naive comparator and it defines the performance limit that can be reached in the absence of any domain adaptation mechanism.

4.4.1 Convolutional Feature Extraction

The backbone of the feature extraction is a set of three consecutive convolutional blocks. CNNBlocks use one-dimensional convolution, batch normalisation, ReLU activation and max-pooling one after another. The channel dimension increases gradually in the three blocks: 3 input channels develop to 32, then to 64, then 128 feature maps. The kernel width of 7 will be used to respond to the short period oscillations of vibration and acoustic emission signals and not to overly overlap the receptive field of neighboring windows. At every step, temporal pooling reduces the length of the sequence such that the LSTM is inputted with a shorter representation with high information density.

4.4.2 LSTM Temporal Encoding and Regression Head

The CNN output is sent to two-layer Long Short-Term Memory network which assumes temporal dependencies in the compressed features sequence. The resulting hidden state of the last LSTM step is then dropped before going through a two-layer linear regression head, to give a single scalar estimate of the flank wear. The dropout offers regularisation to overfitting to the small set of machining instances used in training. The number of trainable parameters of the Baseline is 337,345.

4.5 Material-Agnostic Feature Fusion Model

In this section, the material-agnostic feature fusion model is described.

The Material-Agnostic Feature Fusion Model (MAFFModel) also uses the same CNN backbone as Baseline so that they achieve architectural parity. Domain-Invariant Fusion (DIF) module is placed in-between the CNN output and the LSTM input and works with the 128-channel feature map of the last convolutional block.

4.5.1 Channel Attention Mechanism

Channel Attention sub-module performs a squeeze-and-excitation operation. Global average pooling reduces the time dimension to give a single description on each channel of features. The bottleneck consists of 2 layers; the intermediate layer has the dimensions of 32 and is fully connected with a ReLU activation and computes a 128 dimensional attention weight vector. All weights are restricted to the unit interval by a sigmoid gate. The resultant weights re-calibrate the channel pattern of the feature map, silencing patterns in channels whose pattern of activation is caused by material specific properties, but not by wear state.

4.5.2 Bottleneck Residual Block

An attention-weighted features refinement is performed by a bottleneck residual block. The block itself inverts the channel dimension by the use of a 1x1 convolution, after which the channel dimension is reduced and the reduced representation is subjected to a grouped convolution, and finally inverted again to the channel dimension by a second 1x1 convolution. After every convolution, there comes batch normalisation and ReLU. The skip connection introduces the block input to the output without loss of gradient flow or block representation collapse during deep transformation. The full MAFFModel has 362,657 trainable parameters, which is 7.5 percent overhead over the Baseline.

4.6 Training Configuration

The models are trained using the AdamW optimiser, which incorporates decoupled weight decay to improve generalisation performance. An annealing learning rate schedule decreases the rate gradually by a smooth curve starting with the initial rate to an approximation of zero at the largest possible budget of epochs. Each optimiser step clips gradient norms to a maximum of 1.0 to make training more stable when using short wear-signal windows of large variance. Early stopping monitors validation R^2 is an evaluation of a patience of five consecutive epochs, which recovers the most successful checkpoint before testing. The training loss is measured by mean squared error which is appropriate in accordance with the objective of continuous wear regression.

4.7 Evaluation Metrics

As defined in the experimental framework in Chapter 3, the following evaluation metrics are used. Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE) and coefficient of determination R 2 are the performance measures. The major cross-domain comparison measure is MAPE as it is scale-invariant and scales the deviation as a percentage of the actual wear magnitude and can be interpreted in an industrial context directly. Inference latency is defined as the average and 99 th -percentile single-window prediction time in two hundred consecutive forward passes, in milliseconds.

5. Implementation

5.1 Introduction

The chapter records the execution of the entire experimental pipeline, including setting up of the environment, as well as carrying out all levels of assessment. Each of the stages is characterized by its aim, the major design choices that were made in the code, and how it connects to the architectural and methodological decisions presented in Chapter 3 and Chapter 4. Placeholder in turn refers to code screenshots and figure output to then be included later in the notebook cell screenshots.

5.2 Environment Configuration

The notebook begins by installing the necessary packages that are not in the default Kaggle image such as scipy, scikit-learn, openpyxl, and xlrd. Then all the necessary libraries are loaded with a single import statement: PyTorch, the neural network, its optimiser and data utilities, NumPy to work with arrays, Pandas to work with tabular records, Matplotlib to visualise and SciPy to test statistics. All paths, hyperparameter constants, and random seeds are defined in a single place as a global configuration cell to ensure that no literal values are shown in the downstream code.

```
import subprocess
subprocess.run(['pip', 'install', 'scipy', 'scikit-learn', 'openpyxl', 'xlrd', '--quiet'], check=Tru
```

```
CompletedProcess(args=['pip', 'install', 'scipy', 'scikit-learn', 'openpyxl', 'xlrd', '--quiet'], returncode=
0)
```

Code Snippet 1: Package installation - scipy, scikit-learn, openpyxl, xlrd

Installs required third-party packages not present in the default Kaggle environment.

```python
import os
import io
import copy
import math
import time
import random
import pickle
import zipfile
import warnings
import urllib.request
import numpy as np
import pandas as pd
import scipy.io as sio
from pathlib import Path
from collections import defaultdict

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader, IterableDataset

from sklearn.metrics import mean_absolute_error, r2_score
from scipy import stats

import matplotlib
matplotlib.use('Agg')
import matplotlib.pyplot as plt
import matplotlib.gridspec as gridspec

warnings.filterwarnings('ignore')

SEED = 42
random.seed(SEED)
np.random.seed(SEED)
torch.manual_seed(SEED)
if torch.cuda.is_available():
    torch.cuda.manual_seed_all(SEED)
```

Code Snippet 2: Consolidated import block and global configuration constants

Loads all project libraries and defines global hyperparameter constants and random seeds.

5.3 Data Loading

The UCI Milling data is in the form of a MATLAB.mat binary file. The loader will look to find a local copy of the specified location, otherwise, the file is downloaded at the NASA Prognostics Data Repository and stored locally and then read by the SciPy MATLAB parser. The individual records after the parsing are changed into Python dictionary containing the six raw sensor signal arrays and the corresponding flank wear label array. The runout failures are marked, and separate column filtering out records marked as runout is performed, after which all further processing is restricted to valid progressive-wear cases. QIT-CEMC titanium data is delivered in the form of CSV files per cycle. A custom loader reads a deterministically sorted list of files and reads the three aligned columns of the channel, and appends the VB measure of the metadata file present with the file.

```python
def load_mill_mat(mat_path):
    raw  = sio.loadmat(mat_path)
    mill = raw['mill']
    records = []
    skipped = 0
    for i in range(mill.shape[1]):
        r      = mill[0, i]
        vb_val = float(r['VB'][0, 0])
        if np.isnan(vb_val):
            skipped += 1
            continue
        sig = np.column_stack([r[col].flatten() for col in ALL_SIGNAL_COLS]).astype(np.float32)
        if np.abs(sig).max() > SIGNAL_CLIP:
            skipped += 1
            continue
        records.append({
            'case':    int(r['case'][0, 0]),
            'run':     int(r['run'][0, 0]),
            'vb':      vb_val,
            'feed':    float(r['feed'][0, 0]),
            'doc':     float(r['DOC'][0, 0]),
            'mat':     int(r['material'][0, 0]),
            'signals': sig,
        })
    print(f'Loaded {len(records)} valid records, skipped {skipped}')
    return records

all_records = load_mill_mat(MILL_MAT_PATH)
vbs = [r['vb'] for r in all_records]
print(f'VB range: [{min(vbs):.3f}, {max(vbs):.3f}] mm')
print(f'Cases present: {sorted(set(r["case"] for r in all_records))}')
```

Code Snippet 3: UCI Milling dataset loader - .mat parsing, runout filtering, and record dictionary construction

Parses the UCI Milling .mat file, filters runout records, and constructs per-record signal dictionaries.

```python
def custom_sort_key(filename):
    parts = Path(filename).stem.split('-')
    try:
        return (int(parts[0]), int(parts[1]), int(parts[2]))
    except (IndexError, ValueError):
        return (0, 0, 0)
```

```python
qit_file_pairs = build_qit_file_pairs(
    base_path=QIT_CEMC_BASE,
    force_dir=QIT_FORCE_DIR,
    vib_dir=QIT_VIB_DIR,
    label_filename=QIT_LABEL_FILE,
)
qit_dataset = QITIterableDataset(qit_file_pairs, WINDOW_SIZE, WINDOW_STEP)
qit_loader  = DataLoader(qit_dataset, batch_size=BATCH_SIZE, num_workers=0)

y_qit_labels = np.array([vb for _, _, vb in qit_file_pairs], dtype=np.float32)
print(f'QIT IterableDataset ready — {len(qit_file_pairs)} cycles')
```

Code Snippet 4: QIT-CEMC titanium dataset loader - custom sort key and VB label extraction

Iterates over sorted QIT-CEMC CSV files and extracts aligned sensor channels with corresponding VB labels.

5.4 Exploratory Data Analysis

The integrity of data is ensured by exploratory analysis prior to the process of any modelling. Wear progression curves are generated on a case by case basis of machining and ensure monotonically rising VB curves, which is an assumption of label integrity required by regression. The descriptive statistics of the per-channel are calculated separately in cast iron and steel records in the source dataset to record the differences in amplitude and variance that drive per-window normalisation. Channel selection sweep is plotted to ensure the chosen three channel combination has the largest leave-one-case-out R2 value amongst all the candidate subsets. The domain shift can be observed before any cross-material analysis is done with a VB label distribution plot between the source and target domains.

5.5 Preprocessing Pipeline

Signal segmentation is applied in segmentrecord function. Each signal matrix of a record is swept by a sliding window of constant length and the window has no overlap with the preceding window. Each extracted window is saved together with the wear label of its final sample, the case identifier, and the index of machining run, which is needed to be split stratified and by the few-shot protocol. In case-stratified splitting, each split is made up of all windows, and entire machining cases are allocated to the split to avoid any temporal data leakage. The last step in preprocessing is then the per-window z-score normalisation which computes the mean and standard deviation of the samples of a window itself, eliminating the absolute amplitude bias between the signal in the source and target domain but maintaining the temporal profile, which carries wear state information.

```python
def segment_record(record, window_size, step):
    sig = record['signals']
    T   = sig.shape[0]
    windows = []
    start   = 0
    while start + window_size <= T:
        windows.append({
            'window': sig[start:start + window_size, :],
            'vb':     record['vb'],
            'case':   record['case'],
            'run':    record['run'],
        })
        start += step
    return windows

all_windows = []
for r in all_records:
    all_windows.extend(segment_record(r, WINDOW_SIZE, WINDOW_STEP))

print(f'Total windows : {len(all_windows)}')
print(f'Window shape  : {all_windows[0]["window"].shape}')
```

Code Snippet 5: segment_record windowing function - fixed-length non-overlapping segmentation with label and case identifier assignment

Applies a fixed-length non-overlapping sliding window to each record and attaches wear labels and case identifiers.

```
# Per-window z-score standardisation
# Applied independently per window to remove absolute amplitude differences
# between UCI (volts/dimensionless) and QIT (Newtons/m/s2/Pa) channels

def windows_to_array(windows):
    return np.stack([w['window'] for w in windows], axis=0)

def labels_to_array(windows):
    return np.array([w['vb'] for w in windows], dtype=np.float32)

def per_window_standardise_batch(X):
    mean = X.mean(axis=1, keepdims=True)
    std  = X.std(axis=1, keepdims=True)
    std  = np.where(std < 1e-8, 1e-8, std)
    return ((X - mean) / std).astype(np.float32)

X_train = per_window_standardise_batch(windows_to_array(train_windows))
X_val   = per_window_standardise_batch(windows_to_array(val_windows))
X_test  = per_window_standardise_batch(windows_to_array(test_windows))

y_train = labels_to_array(train_windows)
y_val   = labels_to_array(val_windows)
y_test  = labels_to_array(test_windows)

print(f'X_train: {X_train.shape}  y range: [{y_train.min():.3f}, {y_train.max():.3f}]')
print(f'X_val  : {X_val.shape}    y range: [{y_val.min():.3f}, {y_val.max():.3f}]')
print(f'X_test : {X_test.shape}   y range: [{y_test.min():.3f}, {y_test.max():.3f}]')
```

Code Snippet 6: Per-window z-score normalisation applied independently to train, validation, test, and QIT-CEMC arrays

Computes per-window mean and standard deviation independently for each window and applies z-score standardisation.

5.6 Dataset and DataLoader Construction

WearDataset class is an object that packs the NumPy arrays that have been pre-calculated and placed in a PyTorch Dataset and transpose every window to the channel-first format that is expected by the PyTorch Conv1d operator. DataLoaders are built around the training, validation, and source-domain test splits and the evaluation loaders are shuffled off. The QIT-CEMC data set is processed by an internal IterableDataset which provides the window portions as they are required on-demand via disk, rather than loading the entire set of titanium into the GPU at the same time.

```python
class WearDataset(Dataset):
    def __init__(self, X, y, cases, runs):
        self.X     = torch.from_numpy(X.transpose(0, 2, 1))  # (N, C, W) for Conv1d
        self.y     = torch.from_numpy(y).unsqueeze(1)
        self.cases = torch.from_numpy(cases.astype(np.int32))
        self.runs  = torch.from_numpy(runs.astype(np.int32))

    def __len__(self):
        return len(self.y)

    def __getitem__(self, idx):
        return self.X[idx], self.y[idx], self.cases[idx], self.runs[idx]


train_dataset = WearDataset(X_train, y_train, cases_train, runs_train)
val_dataset   = WearDataset(X_val,   y_val,   cases_val,   runs_val)
test_dataset  = WearDataset(X_test,  y_test,  cases_test,  runs_test)

train_loader = DataLoader(train_dataset, batch_size=BATCH_SIZE, shuffle=True,
                          num_workers=2, pin_memory=True)
val_loader   = DataLoader(val_dataset,   batch_size=BATCH_SIZE, shuffle=False,
                          num_workers=2, pin_memory=True)
test_loader  = DataLoader(test_dataset,  batch_size=BATCH_SIZE, shuffle=False,
                          num_workers=2, pin_memory=True)

print(f'Train batches : {len(train_loader)}')
print(f'Val batches   : {len(val_loader)}')
print(f'Test batches  : {len(test_loader)}')
```

Code Snippet 7: WearDataset class and DataLoader construction

Wraps pre-processed arrays into a PyTorch Dataset and constructs DataLoaders for all data splits.

5.7 Model Architecture Implementation

The entire model classes are formulated within one cell of architecture. One of the convolutional stages is captured by CNNBlock as a nn.Sequential module with Conv1d, BatchNorm1d, ReLU and MaxPool1d. The CNNLSTMBaseline model takes three CNNBlocks with a result output channel width that increases and feeds the output to a two-layered nn.LSTM, and links the last hidden state to regression head with the use of a dropout and two linear layers. ChannelAttention uses the squeeze-and-excitation architecture: adaptive average pooling shrinks the time dimension, two fully connected networks that have an intermediate ReLU and a sigmoid output gate compute the per-channel attention scores, and element-wise multiplication scales the feature map. DomainInvariantFusion is an integration of ChannelAttention and bottleneck residual block into DIF module. DIFMAFFModel Adds DIF between CNN output and LSTM, and backbone and LSTM structures are the same as the Baseline.

```python
class CNNBlock(nn.Module):
    def __init__(self, in_ch, out_ch, kernel=7, pool=2):
        super().__init__()
        self.block = nn.Sequential(
            nn.Conv1d(in_ch, out_ch, kernel_size=kernel, padding=kernel // 2),
            nn.BatchNorm1d(out_ch),
            nn.ReLU(inplace=True),
            nn.MaxPool1d(kernel_size=pool),
        )

    def forward(self, x):
        return self.block(x)
```

```python
class CNNLSTMBaseline(nn.Module):
    def __init__(self, n_channels, cnn_channels, lstm_hidden, lstm_layers, dropout):
        super().__init__()
        self.cnn = nn.Sequential(
            CNNBlock(n_channels,      cnn_channels[0]),
            CNNBlock(cnn_channels[0], cnn_channels[1]),
            CNNBlock(cnn_channels[1], cnn_channels[2]),
        )
        self.lstm = nn.LSTM(
            input_size=cnn_channels[-1],
            hidden_size=lstm_hidden,
            num_layers=lstm_layers,
            batch_first=True,
            dropout=dropout if lstm_layers > 1 else 0.0,
        )
        self.dropout = nn.Dropout(dropout)
        self.head    = nn.Linear(lstm_hidden, 1)

    def forward(self, x):
        feat = self.cnn(x)
        feat = feat.permute(0, 2, 1)
        out, _ = self.lstm(feat)
        return self.head(self.dropout(out[:, -1, :]))
```

Code Snippet 8: CNNBlock, CNNLSTMBaseline class definitions

Defines all five model classes in given cell: CNNBlock, CNNLSTMBaseline, ChannelAttention, DomainInvariantFusion, and MAFFModel.

5.8 Training Loop Implementation

The trainmodel function consists of the training loop. The computation of MSE loss is done on the training batches, gradient norms are clipped, and optimiser is taken by each epoch. Once the model has been tested on the validation loader, the best validation R2 checkpoint will be written to disk. In case validation R^2 has not improved after several consecutive epochs as the patience parameter, training will stop and the trained checkpoint will be restored in advance before resuming. It is the sequential training of both models.

```python
print('=== Training Baseline ===')
history_baseline = train_model(
    model=baseline_model, train_loader=train_loader, val_loader=val_loader,
    num_epochs=NUM_EPOCHS, lr=LR_BASELINE, weight_decay=WEIGHT_DECAY,
    patience=PATIENCE, device=DEVICE, label='Baseline',
)
```

```
=== Training Baseline ===
[Baseline] Epoch   1/60 | loss: 0.0333 | val MAE: 0.1350 | val R2: 0.2786 | best R2: 0.2786 | LR: 5.00e-04
[Baseline] Epoch  10/60 | loss: 0.0071 | val MAE: 0.1019 | val R2: 0.5945 | best R2: 0.6201 | LR: 5.00e-04
[Baseline] Epoch  20/60 | loss: 0.0024 | val MAE: 0.0951 | val R2: 0.6997 | best R2: 0.6997 | LR: 2.50e-04
[Baseline] Epoch  30/60 | loss: 0.0015 | val MAE: 0.1000 | val R2: 0.5979 | best R2: 0.6997 | LR: 1.25e-04
[Baseline] Early stopping at epoch 35
[Baseline] Best val R2: 0.6997
```

Code Snippet 9: Baseline model training call with epoch log showing loss, validation R^2, and early stopping at epoch 35

Executes the Baseline training loop with gradient clipping, cosine LR scheduling, and early stopping at epoch 35.

5.9 Evaluation, Ablation, and Latency Implementation

Training evaluation is done through an evaluate_model utility which executes each model on inference mode and gives per-record metrics. In the source-domain test, window-level predictions are averaged on a per-case basis of machining and then the metrics are computed. In the case of titanium zero-shot evaluation, the window level results are to be compared with window level VB results. Tier 2 ablation The Tier 2 ablation employs a modified version of IterableDataset, which zeroes out one channel at a time before windowing the model, and documenting the change in R2 in each omission. Tier 3 latency This is measured by the measure_latency function, which feeds one dummy window through the model in a loop, and records the wall-clock time per pass.

```
=== Baseline: UCI steel/cast-iron test (cases 12, 15, 16) — record level ===
  MAE: 0.1069
  R2: 0.3183
  MAPE: 31.4242

=== Baseline: QIT-CEMC titanium (zero-shot) ===
  MAE: 0.1258
  R2: -2.2225
  MAPE: 77.6988

MAPE increase steel -> titanium: 147.3% (threshold >35%)
Cross-material degradation confirmed: True

=== MAFF: UCI steel/cast-iron test — record level ===
  MAE: 0.1100
  R2: 0.3400
  MAPE: 37.9177

=== MAFF: QIT-CEMC titanium (zero-shot) ===
  MAE: 0.1561
  R2: -3.5617
  MAPE: 95.6213
```

Code Snippet 10: evaluate_model calls for UCI source-domain test and QIT-CEMC zero-shot transfer - printed MAE, R^2, and MAPE for both models

Runs inference on the UCI test set and QIT-CEMC titanium dataset and prints MAE, R^2, and MAPE for both models.

```python
def measure_latency(model, n_channels, window_size, device, n_runs=200):
    model.eval()
    dummy = torch.randn(1, n_channels, window_size).to(device)
    with torch.no_grad():
        for _ in range(10):
            model(dummy)
    if device.type == 'cuda':
        torch.cuda.synchronize()
    times = []
    with torch.no_grad():
        for _ in range(n_runs):
            t0 = time.perf_counter()
            model(dummy)
            if device.type == 'cuda':
                torch.cuda.synchronize()
            times.append((time.perf_counter() - t0) * 1000)
    return {'mean_ms': np.mean(times),
            'p95_ms':  np.percentile(times, 95),
            'p99_ms':  np.percentile(times, 99)}


lat_baseline = measure_latency(baseline_model, N_CHANNELS, WINDOW_SIZE, DEVICE)
lat_maff     = measure_latency(maff_model,     N_CHANNELS, WINDOW_SIZE, DEVICE)

print('=== Latency (200 single-window inference runs) ===')
print(f'Baseline  mean: {lat_baseline["mean_ms"]:.2f}ms  '
      f'p95: {lat_baseline["p95_ms"]:.2f}ms  p99: {lat_baseline["p99_ms"]:.2f}ms')
print(f'MAFF      mean: {lat_maff["mean_ms"]:.2f}ms  '
      f'p95: {lat_maff["p95_ms"]:.2f}ms  p99: {lat_maff["p99_ms"]:.2f}ms')
print(f'Baseline meets <50ms constraint: {lat_baseline["p99_ms"] < 50}')
print(f'MAFF meets <50ms constraint    : {lat_maff["p99_ms"] < 50}')
```

```
=== Latency (200 single-window inference runs) ===
Baseline  mean: 1.63ms  p95: 1.87ms  p99: 1.92ms
MAFF      mean: 1.77ms  p95: 1.86ms  p99: 2.02ms
Baseline meets <50ms constraint: True
MAFF meets <50ms constraint    : True
```

Code Snippet 11: measure_latency function output - mean and p99 single-window inference latency for Baseline and MAFFModel

Measures single-window inference latency over 200 consecutive forward passes and reports mean and p99 values.

5.10 Few-Shot Adaptation and Statistical Validation

The finetune model operation takes a pretrained model, freezes all convolutional parameters and re-trains the LSTM and regression head with a different small fine-tuning loader at a lower learning rate with a different early stopping budget. The Baseline and MAFFModel are both fine-tuned in the same manner. Statistical validation is then performed in two cells: a Wilcoxon signed-rank test is used to compare per-record absolute errors of the two models on the source-domain test and a bootstrap resampling loop is used to calculate 95 percent confidence intervals on R2 under each of the evaluation conditions.

```
=== Few-shot fine-tuning: Baseline ===
[Baseline-FT] Trainable: 264,321  Frozen (CNN): 73,024
[Baseline-FT] Epoch  1/20 | loss: 0.0004
[Baseline-FT] Epoch  5/20 | loss: 0.0001
[Baseline-FT] Early stopping at epoch 8

=== Few-shot fine-tuning: MAFF ===
[MAFF-FT] Trainable: 289,633  Frozen (CNN): 73,024
[MAFF-FT] Epoch  1/20 | loss: 0.0004
[MAFF-FT] Epoch  5/20 | loss: 0.0001
[MAFF-FT] Early stopping at epoch 9

=== Baseline fine-tuned: QIT titanium test (cycles 11-67) ===
  MAE: 0.1115
  R2: -2.2933
  MAPE: 39.7611

=== MAFF fine-tuned: QIT titanium test (cycles 11-67) ===
  MAE: 0.1120
  R2: -2.3105
  MAPE: 39.9528
```

Code Snippet 12: finetune_model calls for Baseline and MAFFModel with frozen CNN backbone - printed few-shot evaluation metrics

Freezes CNN backbone parameters and fine-tunes LSTM and regression head on ten titanium early-wear cycles.

```python
b_abs_errors = np.abs(b_y_true - b_y_pred)
m_abs_errors = np.abs(m_y_true - m_y_pred)

stat, p_val = stats.wilcoxon(b_abs_errors, m_abs_errors)
print('Wilcoxon signed-rank test — MAFF vs Baseline absolute errors (UCI test records)')
print(f'  n = {len(b_abs_errors)} records')
print(f'  statistic = {stat:.4f}')
print(f'  p-value   = {p_val:.4f}')
print(f'  Significant at alpha=0.05: {p_val < 0.05}')
print(f'  Significant at alpha=0.10: {p_val < 0.10}')
```

```
Wilcoxon signed-rank test — MAFF vs Baseline absolute errors (UCI test records)
  n = 21 records
  statistic = 106.0000
  p-value   = 0.7593
  Significant at alpha=0.05: False
  Significant at alpha=0.10: False
```

```
Bootstrap 95% CI on R2 (2000 resamples):
Condition                                          R2  95% CI
---------------------------------------------------------------------
Baseline — UCI steel test (record-level)        0.3183  [-0.4355, 0.7828]
MAFF     — UCI steel test (record-level)        0.3400  [-0.3812, 0.7676]
Baseline — QIT titanium zero-shot (window)     -2.2225  [-2.2371, -2.2078]
MAFF     — QIT titanium zero-shot (window)     -3.5617  [-3.5809, -3.5409]
Baseline — QIT titanium few-shot (window)      -2.2933  [-2.3000, -2.2864]
MAFF     — QIT titanium few-shot (window)      -2.3105  [-2.3173, -2.3035]
```

Code Snippet 13: Wilcoxon signed-rank test result ($p = 0.7593$) and bootstrap 95% confidence intervals on R^2 across all evaluation conditions

Applies the Wilcoxon signed-rank test and computes bootstrap R^2 confidence intervals across all evaluation conditions.

5.11 Chapter Summary

This chapter has captured the implementation process of the entire pipeline step-by-step, and the mapping of the process to the methodological and architectural decisions outlined in Chapters 3 and 4. Chapter 6 provides an explanation of the implementation of the testing and validation system and the steps implemented to obtain the results in Chapter 7.

6. Testing

6.1 Introduction

In this chapter, the authors outline testing and validation steps undertaken to evaluate and validate the two models in the three-tier evaluation framework, few-shot adaptation protocol, and the statistical validation phase. It is centered on the execution of each test, what a pass or a fail condition is and how the results are gathered to be reported. The numerical results are actually provided in Chapter 7.

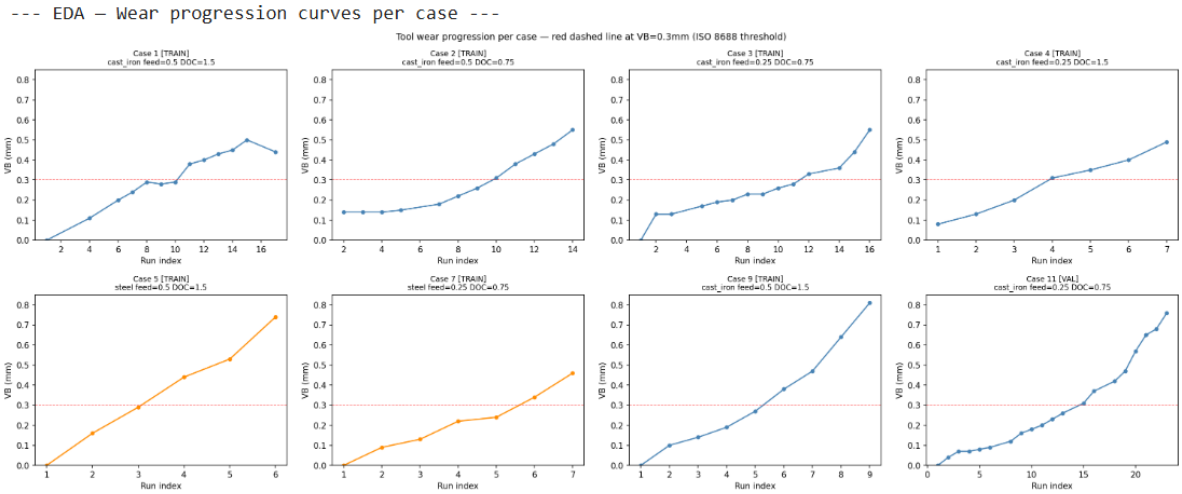

Figure 6.1: Tool wear progression curves per machining case

Wear progression curves *Confirms that flank wear increases monotonically across machining runs in the UCI Milling dataset, validating label integrity for regression-based wear prediction.*

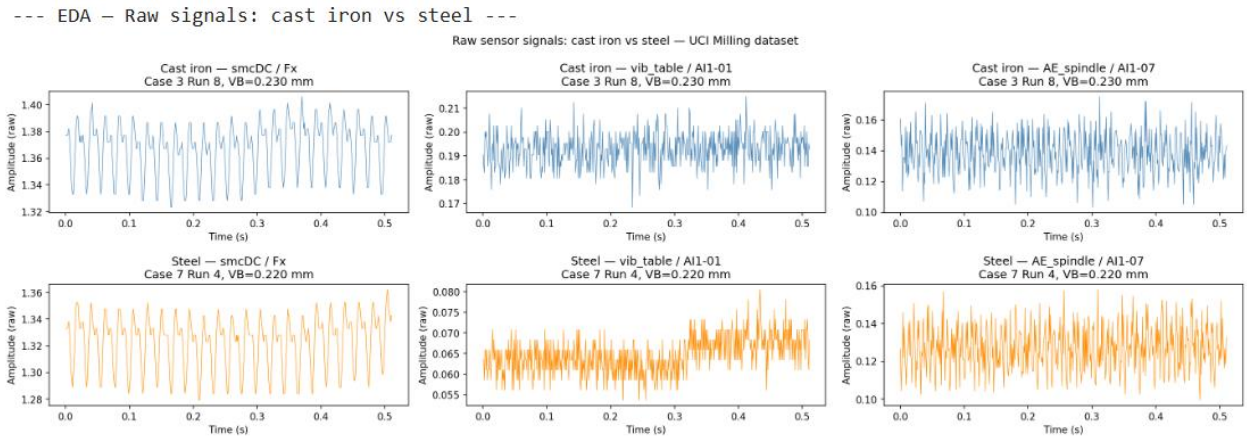

Figure 6.2: Raw signal comparison - cast iron vs steel workpiece

Raw signal comparison (cast iron vs steel) *Illustrates amplitude and pattern differences between cast iron and steel workpiece signals across the three selected sensor channels, motivating per-window normalisation.*

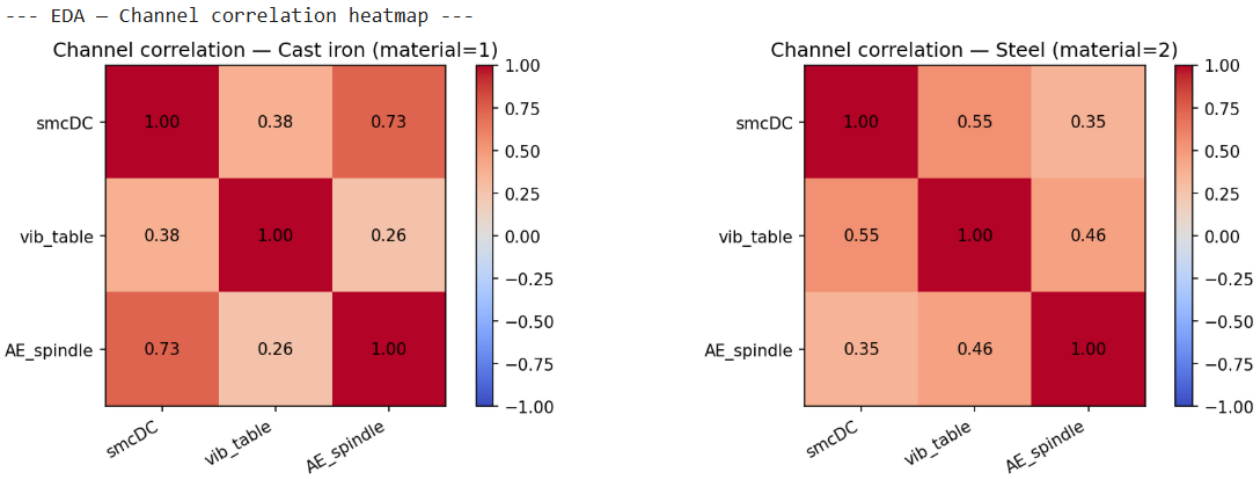

Figure 6.3: Channel correlation heatmap - cast iron vs steel

Channel correlation heatmap *Shows material-dependent inter-channel correlation structures, confirming that signal relationships shift between workpiece materials and supporting the case for domain-invariant feature learning.*

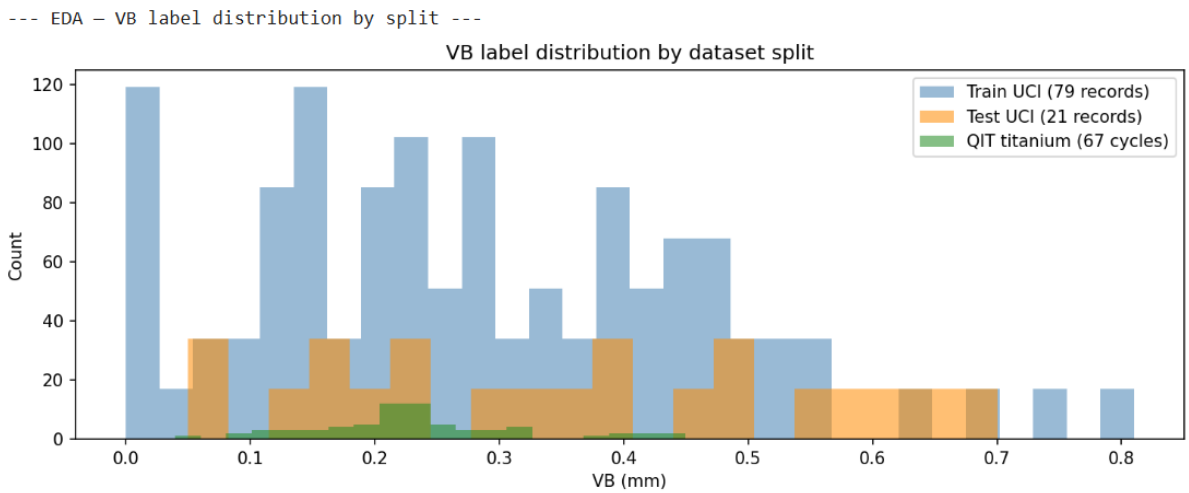

Figure 6.4: VB label distribution - UCI train/test and QIT-CEMC titanium

VB label distribution by split *Compares flank wear value distributions across the UCI training set, UCI test set, and QIT-CEMC titanium dataset, making the cross-material domain shift visible prior to evaluation.*

6.2 Tier 1: Source-Domain Test Execution

After training and model checkpoint restoration, all the models are tested on the held-out source-domain test set of machining cases that were not observed during training or validation. The gradients are turned off and the model is in evaluation mode to make sure that the batch normalisation layers and dropout layers are deterministic. The collection of window-level predictions is done on all the test-set batches and it is subsequently summed up to capture level by averaging all the windows of a given machining case. Averaged predictions are compared to the per-case ground truth values of wear and MAE, RMSE, MAPE and R2 are calculated. It is this level that determines the in-domain ceiling level of performance prior to cross-material transfer determination.

6.3 Tier 1: Zero-Shot Cross-Domain Transfer

The zero-shot evaluation of the source-domain test with no parameter updates is implemented with the same weight of the model on the QIT-CEMC titanium dataset. The habit IterableDataset reads every titanium cycle file in order, and z-score normalises the windows using statistics of only that window, then sends the normalised windows to the model. The predictions are not aggregated to cycle level but instead to window level because the VB values per cycle in the titanium dataset are the values of a single machining run and not a machining case. Cross-material domain shift is established through a pre-determined MAPE degradation threshold with respect to performance in the source domain.

6.4 Tier 2: Signal Ablation Execution

The signal ablation experiment examines what sensor channels can be used to predict cross-material. It uses a custom version of IterableDataset which takes a parameter of ablation channel index; on iteration, it sets all the values of the specified channel to zero and returns the window. Ablation runs are run on three different channels of choice which are 3. The comparison of the R2, MAE, and MAPE values of each of the ablation conditions to the unablated zero-shot baseline are used to determine whether the contributions of each channel can be distinguished. The findings of this level guide the decision of whether the three signal modalities are differentially weighted by the domain-invariant fusion mechanism.

6.5 Tier 3: Latency Verification

The viability of deployment is confirmed through the time that single-window inference takes. The measure_latency function creates a one sample dummy tensor of the appropriate shape, locates it on the same device as the model and measures wall clock time on two hundred consecutive forward passes during a warm loop. The initial ten iterations are used as a warm-up to eliminate cold-start initialisation bias in the GPUs. The mean latency and the 99th-percentile latency are given in

milliseconds. The pass criterion is that the 99th-percentile latency should be less than 50 ms per window which is the operating limit of a real-time implementation in a CNC milling environment.

6.6 Few-Shot Adaptation Execution

The few-shot protocol questions whether a tiny number of target-domain labelled cycles can be adequate to rediscover useful wear prediction on a domain shifted model. These machining cycles of the titanium dataset are the first to reach the fine-tuning set and the rest of the machining cycles are the held-out test set. The convolutional parameters remain fixed and the weights of LSTM and regression head are the only ones updated. Another early stopping condition observes the fine-tuning loss using a lower patience to avoid overfitting to the fine-tuning set. Once the adaptation models have been tuned, both of the adapted models are tested on the titanium held-out set with the same window-level metric computation as the zero-shot tier.

6.7 Statistical Validation Execution

The results obtained are statistically tested by using two tests. Wilcoxon signed-rank test is used to compare the per-record absolute errors of the Baseline and MAFFModel on the source-domain test set. The reason behind the selection of this non-parametric test is that distributional assumptions cannot be assessed reliably in cases where the sample size of the test records is small. The two tailed significant level of 0.05 is used. Bootstrap confidence intervals are then calculated on R2 under each of the evaluation conditions with 2,000 resampling draws. The intervals measure the uncertainty in estimating the true value because of the small size of the test samples and are presented together with point estimates in Chapter 7.

7. Results and Evaluation

7.1 Introduction

In this chapter, the entire findings of all levels of evaluation, the few-shot adaptation test, and the statistical validation are shown. The CNNLSTMBaseline and the MAFFModel are reported to have results in each condition. Chapter 8 will focus on the discussion and interpretation of the findings.

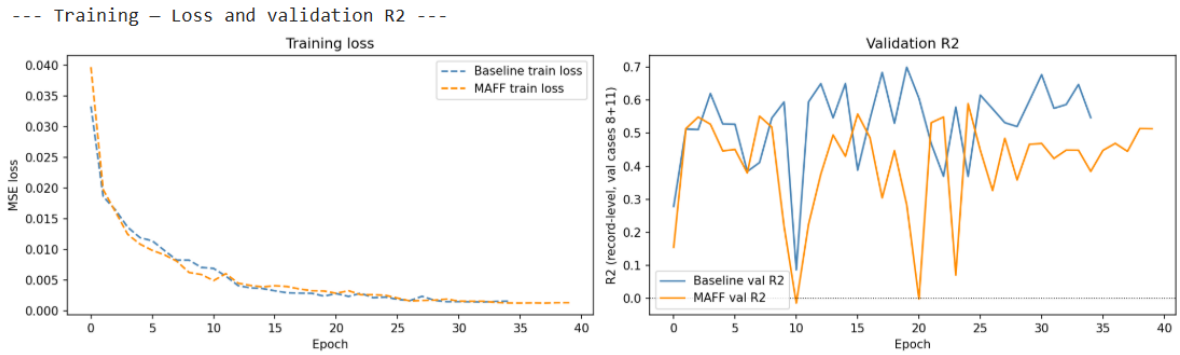

Figure 7.1: Training and validation loss curves - Baseline and MAFFModel

Training and validation loss curves *Shows convergence behaviour for both models over training epochs, with early stopping triggered before the maximum epoch budget in both cases.*

7.2 Source-Domain Performance (Tier 1 In-Domain)

The held-out source-domain test set of machining cases which were not seen during training was used to evaluate both models. Record-level performance measures are reported in table 7.1. The Baseline had an R2 of 0.3183 and MAE of 0.1069 mm and MAPE of 31.42 percent. The MAFFModel had slightly better R2 value of 0.3400 and MAE value of 0.1100 mm and MAPE value of 37.92 percent. The similarity between the two models is that both show a moderate predictive performance on in-domain data, with slightly higher MAPE of the MAFF model suggesting a wider spread on its per-case errors although it has a higher R2. The Baseline and MAFFModel had validation R2 at best checkpoint values of 0.6997 and 0.5894 respectively indicating that both models overfit between validation and held-out test distribution to some extent.

Model	**R^2**	**MAE (mm)**	**MAPE (%)**
CNNLSTMBaseline	0.3183	0.1069	31.42
MAFFModel	0.3400	0.1100	37.92

Table 7.1: Source-domain test performance (record-level, UCI steel and cast iron)

7.3 Zero-Shot Cross-Domain Transfer (Tier 1 Cross-Domain)

Under zero-shot transfer to QIT-CEMC titanium, both models exhibited severe performance degradation. The Baseline produced an R^2 of −2.2225 and a MAPE of 77.70 percent, confirming that the source-domain model systematically mis-predicts titanium wear at the window level. The MAFFModel degraded further to an R^2 of −3.5617 and a MAPE of 95.62 percent. Both results are substantially below the pre-defined MAPE degradation threshold, confirming that cross-material domain shift is severe under the zero-shot condition. The MAFFModel's worse performance relative to the Baseline at this stage indicates that the DIF module, as trained solely on source-domain data, does not produce representations that transfer better to titanium without any target-domain exposure.

Model	**R^2**	**MAE (mm)**	**MAPE (%)**
CNNLSTMBaseline	-2.2225	0.1258	77.70
MAFFModel	-3.5617	0.1561	95.62

Table 7.2: Zero-shot titanium performance (window-level, QIT-CEMC)

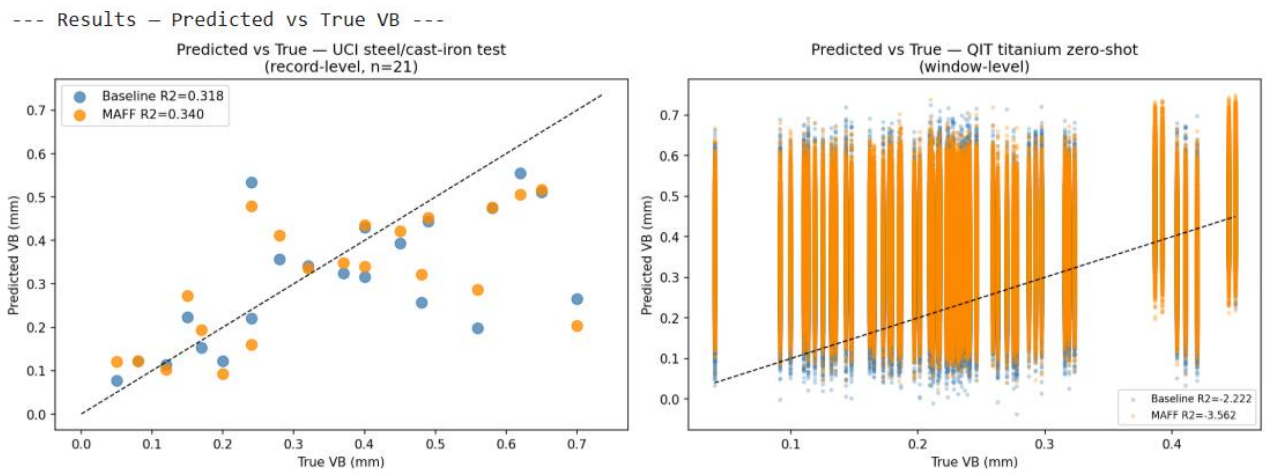

Figure 7.2: Predicted vs actual flank wear - steel test and titanium zero-shot

Predicted vs actual VB *Compares model predictions against ground truth wear values on the steel test set and titanium zero-shot set, visually confirming the severity of cross-material prediction failure.*

7.4 Signal Ablation Results (Tier 2)

The signal ablation study evaluated the effect of zeroing each of the three sensor channels independently during titanium zero-shot inference. In both models, there was no significant difference in the R 2, MAE or MAPE between the zeroing a single channel and the zeroing of the others when compared with the condition of zero-shot. This finding means that in extreme domain shift, no single channel has a decisive role as the domain shift is universal to all the three signal modalities and none of the channels has enough wear-related signal when measured on titanium without adaptation. The failure to generalise is shown by the ablation to be a systemic cross-material effect and not an effect of any one sensor becoming uninformative.

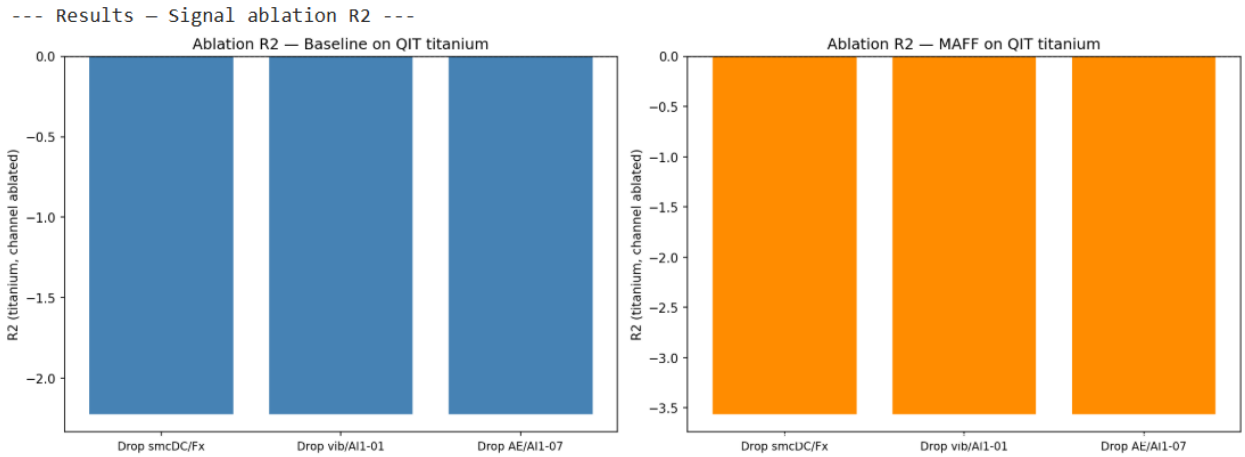

Figure 7.3: Signal ablation R^2 - effect of channel removal on titanium inference

Signal ablation R^2 *Displays the change in R^2 when each sensor channel is individually zeroed during titanium inference, showing no channel acts as a decisive cross-material discriminator.*

7.5 Latency Results (Tier 3)

Table 7.3 reports inference latency measured over two hundred single-window forward passes. Both models satisfy the sub-50 ms per-window deployment constraint by a large margin. The Baseline mean latency of 1.63 ms and p99 of 1.92 ms reflect its smaller parameter count. MAFFModel mean latency of 1.77 ms and p99 of 2.02 ms represent a modest overhead attributable to the DIF module's attention and residual computations. Both models are viable for real-time deployment in a CNC milling context.

Model	**Mean (ms)**	**p95 (ms)**	**p99 (ms)**	**< 50 ms constraint**
CNNLSTMBaseline	1.63	1.87	1.92	Pass

MAFFModel	1.77	1.86	2.02	Pass

Table 7.3: Inference latency (200-run measurement, GPU)

7.6 Few-Shot Adaptation Results

Following the freezing and fine-tuning of CNN backbone, LSTM, and regression head on ten initial wear titanium cycles, the two models were re-tested on the held-out titanium test set. The results are presented in Table 7.4, which compares them to the values of pre-adaptation zero-shot. The Baseline few-shot R^2 dropped from −2.2225 to −2.2933, which is a slight degradation of 0.07 R^2 units whereas MAPE improved significantly, dropping to 39.76 percent as compared to 77.70 percent. The MAFFModel few-shot R2 was now improved by −3.5617 to −2.3105, (a recovery of 1.25 units of R^2) and MAPE has converged to 39.95 percent. The MAPE of the two models became almost the same upon adaptation. This high R2 recovery of MAFFModel, although it has worse initial performance on the zero-shot baseline, is possibly due to the fact that the DIF module offers a feature representation that is more gradient-responsive on a small sample of the target domain.

Model	**Condition**	**R^2**	**MAE (mm)**	**MAPE (%)**
CNNLSTMBaseline	Zero-shot	-2.2225	0.1258	77.70
CNNLSTMBaseline	10-cycle FT	-2.2933	0.1115	39.76
MAFFModel	Zero-shot	-3.5617	0.1561	95.62
MAFFModel	10-cycle FT	-2.3105	0.1120	39.95

Table 7.4: Few-shot adaptation results vs zero-shot baseline (titanium test, window-level)

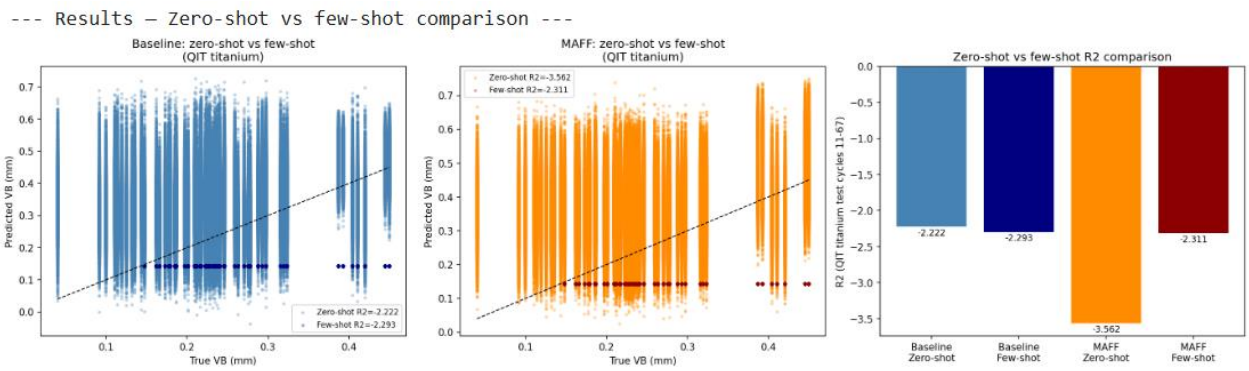

Figure 7.4: Zero-shot vs few-shot adaptation comparison - both models

Zero-shot vs few-shot comparison *Contrasts model predictions before and after ten-cycle fine-tuning on titanium data, illustrating the MAPE recovery achieved by both models through few-shot adaptation.*

7.7 Statistical Validation

The Wilcoxon signed-rank test comparing per-record absolute errors between both models on the source-domain test produced a p-value of 0.7593, substantially above both the 0.05 and 0.10 significance thresholds. The null hypothesis that the two models produce equivalent absolute errors on the source-domain test cannot be rejected. This result is consistent with the near-identical MAPE values produced by the two models on the source-domain test. Bootstrap 95 percent confidence intervals on R^2 for the source-domain test are wide, spanning from approximately −0.44 to 0.78 for the Baseline and −0.38 to 0.77 for MAFFModel, reflecting the small number of test records. Titanium confidence intervals are narrow due to the larger window count but are centred on strongly negative values, confirming the systematic nature of the cross-material prediction failure.

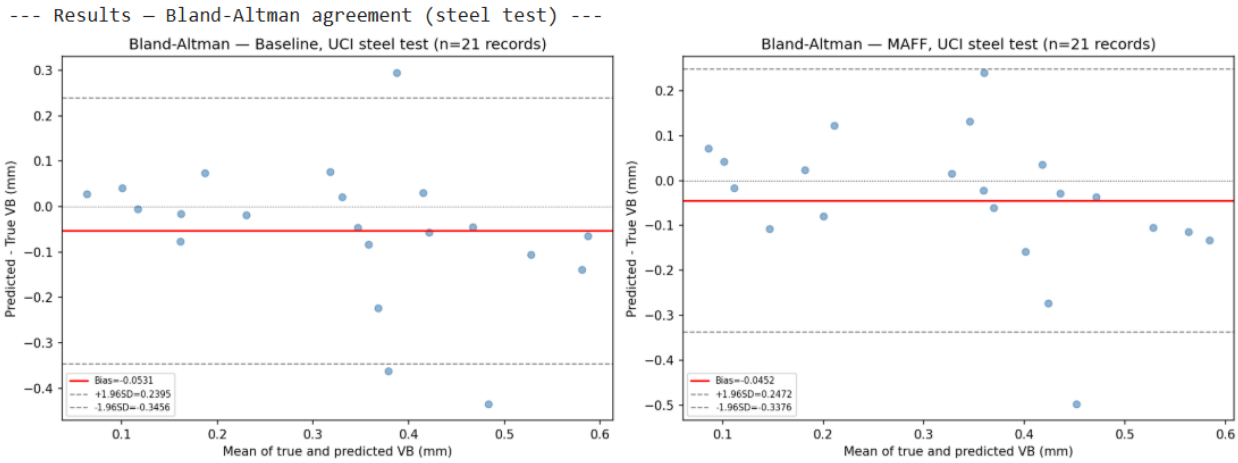

Figure 7.5: Bland-Altman agreement plots - source-domain test

Bland-Altman agreement plots *Plots mean prediction against prediction error to reveal systematic bias and limits of agreement for both models on the source-domain test set.*

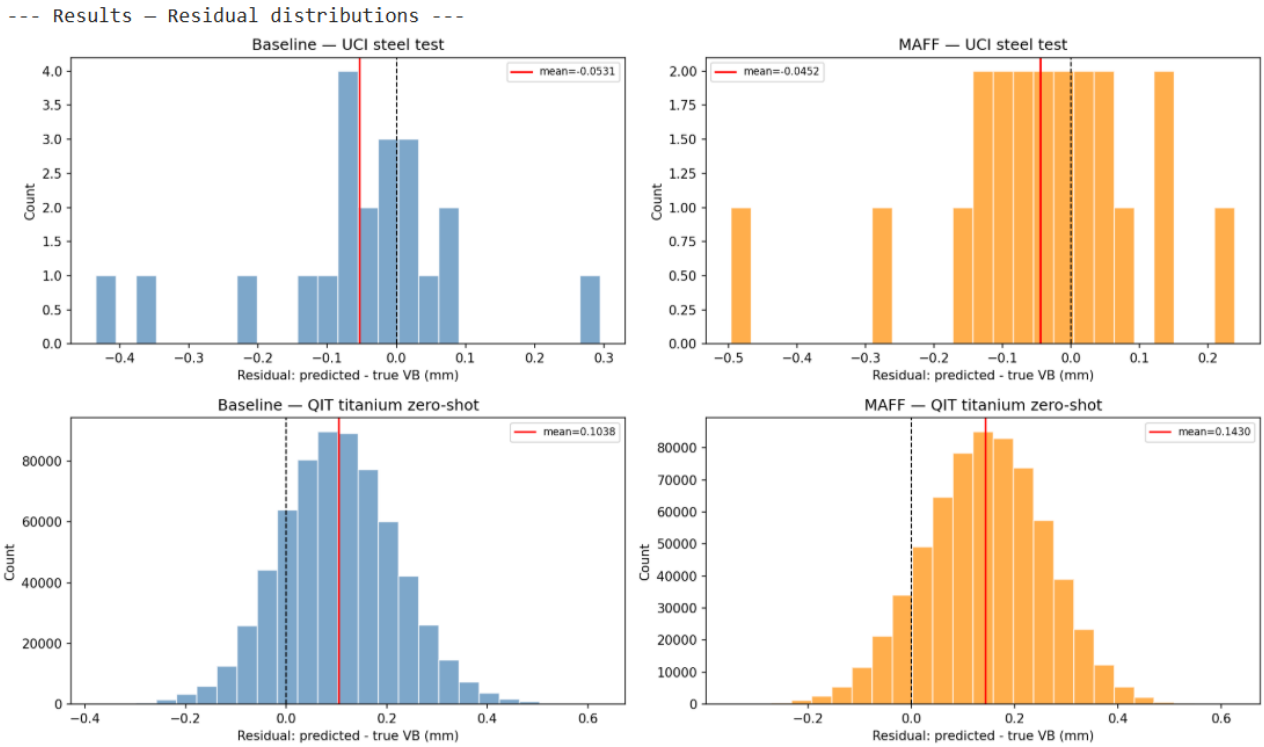

Figure 7.6: Residual distributions across evaluation conditions

Residual distributions *Shows the distribution of prediction errors across evaluation conditions, used to assess zero-mean error assumption and approximate normality for statistical validation.*

7.8 Summary of Findings

The findings show that the two models are both moderate in-domain wear prediction but fail in a categorical mode of prediction at zero-shot cross-material transfer. The DIF module of the MAFFModel does not result in a zero-shot edge over the Baseline; but it offers a representational base upon which recovery of R2 can be attained significantly greater following the slightest target-domain fine-tuning. The obtained results of latency indicate that both architectural complexity and the DIF overhead do not negatively affect the viability of real-time deployment. Statistical testing does not identify any significant source-domain difference of performance between the two models.

8. Discussion and Reflection

8.1 Introduction

This chapter interprets the results from Chapter 7 in relation to the research objectives. It evaluates the effectiveness of the material-agnostic feature fusion approach, identifies limitations, suggests future work, and reflects on ethical considerations.

8.2 Interpretation of Major Findings

Neither of the two models achieves zero-shot cross-material transfer, which validates that there is notable domain shift between materials such as steel and titanium. The MAFF model does not perform well in zero-shot, suggesting that the source-domain bias is strengthened by the attention mechanisms. Nevertheless, MAFF recovers quicker in few-shot tasks, indicating its effective capability of minimising the necessary target-domain data instead of complete generalisation.

8.3 Research Contribution

The paper presents an organized evaluation system incorporating in-domain evaluation, zero-shot transfer, and few-shot adaption. It offers an empirical evidence that feature fusion enhances adaptability, as opposed to actual domain invariance, as a part of the comprehension of cross-material learning in TCM.

8.4 Limitations

The shortcomings are small size of dataset, limited variety of materials, and absence of domain adversarial training. Per-window normalisation eliminates amplitude information and only experiments are carried out using offline datasets without industry validation.

8.5 Future Work

Future directions include adversarial domain learning and application to more materials, and online adaptation techniques. Such strategies can enhance the generalisation and decrease the reliance on labelled data.

8.6 Ethical Reflection

The data in the study is non-sensitive and has few ethical issues. But in practice, it would need to be validated to be safe, controlled, and meet industrial standards.

8.7 Critical Analysis of Model Limitations

The models do not perform zero-shot transfer and are reliant on target-domain data. Poor sensor diversity, overfitting and interpretability also limit performance. The cross-material generalisation is not resolved yet and needs the development of sophisticated domain adaptation.

9. Project Management

9.1 Overview

This chapter presents the project management strategy that was used in the development of the machine learning-based tool condition monitoring (TCM) system. There was a well-organized and milestone-based plan that had to be followed to guarantee that the research goal could be accomplished on time within the academic schedule. The project has focused on successful planning, iterative development, and continuous evaluation in line with the overall research approach.

9.2 Project Timeline and Planning

The project was planned into sequential stages within a period of about 12-14 weeks. The first stage was concerned with the definition of the problem, literature review, and learning about machining data of steel and titanium. This was preceded by data preprocessing, signal analysis and channel selection. The development step entailed the construction of the baseline CNN-LSTM model and the proposed Material-Agnostic Feature Fusion (MAFF) model. The next steps were experimental assessment, such as zero-shot transfer, signal ablation, and few-shot adaptation. The last phase was dedicated to the result analysis, discussion, and the writing of a dissertation. The milestone-based schedule helped in ensuring that every phase was done according to the schedule.

9.3 Meetings and Integration of Supervisory Feedback

Frequent supervisory meetings were also made during the course of the project to check the progress and to narrow the research direction. Feedback was important to enhance the scope and technical depth of the study.

The study started with a general approach to predictive maintenance but narrowed down to tool condition monitoring with a particular focus on cross-material generalisation. The additional direction resulted in the addition of domain-invariant feature fusion and structured testing methods including zero-shot and few-shot testing. Feedback also enhanced the experimental design, evaluation measures and general academic rigor of the project.

9.4 Risk Management

There were many risks that were identified and mitigated in the project. Cross-material domain shift was a significant challenge that had a significant impact on the performance of the models. This was alleviated with the addition of few-shot adaptation and domain-focused evaluation.

The dropout, early stopping and validation monitoring were used to manage risks of overfitting. Preprocessing and window-based normalisation minimised data-related risks, including imbalance and noise. Computation was managed with GPU-based environments and effective data loading methods. Such strategies guaranteed stability and reliability of models.

9.5 project execution and monitoring

Its development was carried out in an iterative manner with every new step being continuously tested and improved. Progress monitoring every week was used to make sure that a deadline is followed and that the problem can be identified early.

There was intermediate validation at all stages such as the preprocessing of data, training of the model and evaluation to avoid the propagation of errors. The combination of theoretical design and experimental findings guaranteed congruency with the aim of research. This organized process of work allowed to utilize time and resources effectively and produce quality output.

10. Conclusion

10.1 Introduction

The present project explored the question of whether a material-agnostic feature fusion method is applicable in reducing the generalisation gap in tool condition monitoring (TCM) when a model is trained using steel/cast-iron milling and implemented on titanium alloy milling. Two deep learning models have been designed and tested: a CNN-LSTM control and the novel Material-Agnostic Feature Fusion (MAFF) model that includes a domain-invariant fusion operation. The driving force was the practical need that wear data labelling is an expensive exercise per new material, and an efficient cross-material strategy will save **retraining and** enhance scalability of predictive maintenance systems.

10.2 Summary of Research

The design-and-evaluate method was strictly followed in the work. The first step was to choose two datasets that are indicative of various material regimes to form a robust cross-material shift between the source and target space. A common preprocessing pipeline was used which involved the fixed window segmentation with per-window normalisation used to reduce the influence of amplitude bias whilst preserving the time-related signatures of wear. The experiment also used two similar models with the same CNN feature extractor and LSTM temporal encoder, so that the difference in performance could not be explained by the different capacity of the models.

The assessment was structured around a multi-tier structure: (i) source-domain in-domain testing, (ii) zero-shot cross-domain transfer (steel-titanium with no parameter updates), (iii) signal ablation to gain insights into the reliance of each modality, (iv) latency testing to ensure that the test is feasible in real-time and (v) few-shot adaptation with using limited labelled target cycles

10.3 Key Findings

To begin with, the performance of both models in the held-out source-domain test set has been modest with the R^2 of about 0.32-0.34 and a MAPE of about 31-38% respectively. Specifically, the baseline achieved $R^2 = 0.3183$ and MAPE = 31.42%, while MAFF achieved $R^2 = 0.3400$ and MAPE = 37.92%. The difference between validation and test behaviour meant that it was somewhat overfitted or not matching the distribution even in the source domain.

Second, both models suffered in performance when transferred to titanium in a condition of strict zero-shot transfer, validating the fact that cross-material domain shift is aggressive and non-random. The baseline produced $R^2 = -2.2225$ and MAPE = 77.70%, while MAFF degraded further to $R^2 = -3.5617$ and MAPE = 95.62%. This demonstrates that the suggested fusion module when trained solely on the source material failed to form representations that generalise to titanium having no target exposure.

Third, the signal ablation outcomes implied that none of the sensor modalities seemed to play a decisive role when cross-material failure occurred; rather, the domain shift influenced the overall signal -wearing correlation between modalities. That is, failure was not caused by any channel being rendered useless, but rather by the whole input-to-wear mapping shifting among materials.

Fourth, both models were easily within real-time limits. The mean latency and MAFF baseline were 1.63 ms and 1.77 ms, respectively, and the p99 latency was close to the mark of about 2 ms, which is much less than a sub-50 ms limit. This means that the complexity of models is not restrictive to the real time deployment, generalisation is.

Lastly, least number of shot adaptation enhanced performance behaviour, particularly the percentage error. Following fine-tuning of ten initial titanium cycles, MAPE dropped to approximately ~40% in both models, and MAFF displayed a more robust recovery trend than its deplorable zero-shot initial condition (R 2 increased to -3.5617 -2.3105). Nevertheless, R2 values were negative suggesting that the chosen adaptation was inadequate to achieve reliable titanium prediction, although it demonstrated that some information in the target domain assists performance recovery in a more efficient manner than zero-shot transfer

10.4 Answer to Research Question

The results indicate that zero-shot cross-material prediction cannot be done using the proposed method. Nevertheless, it is possible to adapt with few-shot efficiency, which implies that the target-domain data is required to be limited to achieve good performance.

10.5 Contributions of Study

The work is original in that it presents clear experimental results that domain shift can severely affect cross-material tool-wear prediction models, such that those trained on steel/cast iron perform disastrously when applied to titanium in a strict zero-shot test. It also provides a reproducible evaluation framework that is more comprehensive than in-domain accuracy by integrating in-domain testing, zero-shot transfer, signal ablation, verification of latency, few-shot adaptation, and statistical verification, which simplifies the comparison and diagnosis of model behaviour. The work also applies a practical implementation of a Material-Agnostic Feature Fusion (MAFF) architecture as a CNNLSTM pipeline and shows that the fusion can be added at negligible real-time cost, without violating strict latency limits. Lastly, by demonstrating that ablation does not fully deactivate a single, dominating, sensor channel and that limited target data offers better recovery, the study provides practical information: enhancing cross material TCM might need enhanced domain generalisation/adaptation interventions instead of sensor specific remedies or domain directed, pure tuning.

10.6 Final Remarks

On the whole, the research confirms one idea: the standard deep learning pipelines do not allow predicting tool wear, cross-material and zero-shot, even with feature fusion. The thermo-mechanical change in cutting dynamics which occurs as a result of the shift between steel/iron and titanium domains is sufficiently altered that thermo-mechanical models trained on one regime fail to predict the other in a systematic manner. Meanwhile, the project shows that the models are computationally feasible to be used in real-time and that performance can be partially restored when little data is available in the target-domain.

Consequently, future systems that would like to be material-agnostic TCM need to focus on explicit domain generalisation or adaptation mechanisms (not just fusion) and need to consider a small quantity of target calibration data as realistic and not exceptional. This direction is more in keeping

with the industrial constraints in which a small amount of labelled target wear can be collected, whereas large amounts of target data to retrain the entire model are not

References

Abraham, B.M. and Gogotsi, Y. (2026) 'Machine learning toolkits and frameworks for materials design,' Wiley Interdisciplinary Reviews Computational Molecular Science, 16(2). https://doi.org/10.1002/wcms.70067

Prodhan, R.K., Fazle, A.B. and Islam, M.M. (2023) 'AI-powered predictive failure analysis in pressure vessels using real-time sensor fusion: enhancing industrial safety and infrastructure reliability,' American Journal of Scholarly Research and Innovation, 02(02), pp. 102-134. https://doi.org/10.63125/wk278c34

Bobbili, V.S.R. et al. (2025) 'Physics-informed neural networks for predicting high-strain-rate energy absorption in additively manufactured lattice materials,' Progress in Additive Manufacturing, 11(2), pp. 2095-2121. https://doi.org/10.1007/s40964-025-01460-3

Cevik, Z.A. and Sariisik, G. (2025) 'Machine Learning-Driven optimization of computer numerical control milling parameters for enhanced surface integrity and burr control in additive manufacturing,' Journal of Tribology, 148(2). https://doi.org/10.1115/1.4069030

De Leon, E. et al. (2025) 'A review of modeling, simulation, and process qualification of additively manufactured metal components via the Laser Powder Bed Fusion method,' Journal of Manufacturing and Materials Processing, 9(1), p. 22. https://doi.org/10.3390/jmmp9010022

Fan, Y. (n.d.) Latent Representations and Fusion Techniques for Probabilistic Structural Health Monitoring and State Awareness. ProQuest. Available at: https://www.proquest.com/openview/d43bd3a0a0d15f6dc519777258a14133/1

Gao, J. et al. (2025) 'Generative adversarial network-enabled microstructural mapping from surface profiles for laser powder bed fusion,' Virtual and Physical Prototyping, 20(1). https://doi.org/10.1080/17452759.2025.2499925

Hassan, M.A. et al. (2025) 'Focus to generalize (F2G): physics-guided attention for sample efficient and generalizable deep learning defect detection,' Journal of Intelligent Manufacturing. https://doi.org/10.1007/s10845-025-02623-3

Kaur, R., Kumar, R. and Aggarwal, H. (2025) 'Systematic review of Artificial intelligence, machine learning, and deep learning in machining operations: advancements, challenges, and future directions,' Archives of Computational Methods in Engineering, 32(8), pp. 4983-5036. https://doi.org/10.1007/s11831-025-10290-z

Kishor, G. et al. (2026) 'A review on in-situ monitoring, process control and residual stress prediction using machine learning methods in metal-based laser additive manufacturing,' Progress in Additive Manufacturing. https://doi.org/10.1007/s40964-026-01532-y

Kjer, M.B. et al. (2024) 'Applying systems engineering principles to develop an open-source laser based metal powder bed fusion system,' Rapid Prototyping Journal, 30(9), pp. 1911-1928. https://doi.org/10.1108/rpj-12-2023-0422

Kollu, G.S. and Kittur, J. (2025) 'Integrating Deep Learning into Mechanical Engineering: A Systematic Review of applications and Educational implications,' Computer Applications in Engineering Education, 33(3). https://doi.org/10.1002/cae.70048

Kusoglu, I.M. et al. (2025) 'Large-Scale Interlaboratory Study Along the Entire Process Chain of Laser Powder Bed Fusion: Bridging Variability, Standards, and Optimization across Metals and Polymers,' Advanced Engineering Materials, 27(14). https://doi.org/10.1002/adem.202402930

Lach, L. and Svyetlichnyy, D. (2026) 'Advanced numerical modeling of powder bed fusion: From Physics-Based simulations to AI-Augmented Digital Twins,' Materials, 19(2), p. 426. https://doi.org/10.3390/ma19020426

Lehmann, T. et al. (2021) 'Large-scale metal additive manufacturing: a holistic review of the state of the art and challenges,' International Materials Reviews, 67(4), pp. 410-459. https://doi.org/10.1080/09506608.2021.1971427

Liang, X. et al. (2025) 'Machine-Learning-Guided design of nanostructured metal oxide photoanodes for photoelectrochemical water splitting: From material discovery to performance optimization,' Nanomaterials, 15(12), p. 948. https://doi.org/10.3390/nano15120948

Liu, Y. et al. (2025) 'Monitoring and Control of the Direct Energy Deposition (DED) additive manufacturing process using Deep learning techniques: A review,' Materials, 19(1), p. 89. https://doi.org/10.3390/ma19010089

Long, T. et al. (2025) 'Recent progress of artificial intelligence application in polymer materials,' Polymers, 17(12), p. 1667. https://doi.org/10.3390/polym17121667

Madika, B. et al. (2025) 'Artificial intelligence for materials discovery, development, and optimization,' ACS Nano, 19(30), pp. 27116-27158. https://doi.org/10.1021/acsnano.5c04200

Meng, Y. et al. (2024) 'Meta-Learning-Based domain generalization for Cost-Effective tool condition monitoring in ultrasonic metal welding,' IEEE Transactions on Industrial Informatics, 21(1), pp. 653-662. https://doi.org/10.1109/tii.2024.3456671

Naeim, B. et al. (2025) 'Machine learning approaches for fatigue life prediction of steel and feature importance analyses,' Infrastructures, 10(11), p. 295. https://doi.org/10.3390/infrastructures10110295

Nazir, Q. and Shao, C. (2021) 'Online tool condition monitoring for ultrasonic metal welding via sensor fusion and machine learning,' Journal of Manufacturing Processes, 62, pp. 806-816. https://doi.org/10.1016/j.jmapro.2020.12.050

Omar, I., Khan, M. and Starr, A. (2023) 'Suitability analysis of machine learning algorithms for crack growth prediction based on dynamic response data,' Sensors, 23(3), p. 1074. https://doi.org/10.3390/s23031074

Papatheodorou, A. et al. (2025) 'Recent Advances in Sensor Fusion monitoring and control Strategies in Laser Powder Bed Fusion: A review,' Machines, 13(9), p. 820. https://doi.org/10.3390/machines13090820

Rane, N. (2023) 'Transformers in Material Science: Roles, challenges, and future scope,' SSRN Electronic Journal. https://doi.org/10.2139/ssrn.4609920

Saatci, B.T., Ulas, M. and Gurgenc, T. (2025) 'Hybrid AI Systems for tool wear Monitoring in Manufacturing: A Systematic review,' Applied Sciences, 16(1), p. 208. https://doi.org/10.3390/app16010208

Shao, J. et al. (2025) 'Multi-source feature augmentation adversarial domain generalization method for tool wear condition monitoring on unseen machine tools,' Journal of Intelligent Manufacturing. https://doi.org/10.1007/s10845-025-02685-3

Sheikh, S. et al. (2025) 'High-throughput alloy and process design for metal additive manufacturing,' Npj Computational Materials, 11(1), p. 179. https://doi.org/10.1038/s41524-025-01670-x

Shin, S. et al. (2023) 'Material-Adaptive anomaly detection using Property-Concatenated transfer learning in wire ARC additive manufacturing,' International Journal of Precision Engineering and Manufacturing, 25(2), pp. 383-408. https://doi.org/10.1007/s12541-023-00924-2

Stephen, L.K.H. (2025) Real-time digital twin and machine learning solutions for hole quality and tool condition monitoring in robotic drilling of composite materials. IEEE. https://doi.org/10.34961/researchrepository-ul.28632179

Wang, L. et al. (2025) 'Porosity defects in additively manufactured metal materials: Formation mechanisms, impact on performance and regulation,' International Materials Reviews. https://doi.org/10.1177/09506608251371459

Wang, W. et al. (2026) 'Visible-Infrared Dual-Modal monitoring system for overlap defects in wire arc additive manufacturing,' Materials, 19(5), p. 899. https://doi.org/10.3390/ma19050899

Ye, L. et al. (2025) 'Review of online quality control for laser directed energy deposition (LDED) additive manufacturing,' International Journal of Extreme Manufacturing, 7(6), p. 062005. https://doi.org/10.1088/2631-7990/aded4f

Zamzu, C.O., Jia, Z. and Shen, B. (2025) 'A mini review on evolution of High-Entropy alloy design: From experimental approaches to machine learning integration,' Rare Metals, 45(1). https://doi.org/10.1002/rar2.70048

Zhang, J. et al. (2025) 'Toward digital twins for intelligence manufacturing: Self-adaptive control in assisted equipment through multi-sensor fusion smart tool real-time machine condition monitoring,' Journal of Manufacturing Systems, 82, pp. 301-318. https://doi.org/10.1016/j.jmsy.2025.06.020

Zhou, H. et al. (2025) 'Machine learning-driven material intelligence research and development,' Nano Research, 18(12), p. 94908095. https://doi.org/10.26599/nr.2025.94908095

Zhu, K., Fuh, J.Y.H. and Lin, X. (2021) 'Metal-Based Additive Manufacturing Condition Monitoring: A review on machine learning based approaches,' IEEE/ASME Transactions on Mechatronics, 27(5), pp. 2495-2510. https://doi.org/10.1109/tmech.2021.3110818

APPENDIX A- Source Code Snippets

A.1 Data Loading and Dataset Integration

```
# Load UCI Milling dataset
mill_data = loadmat("mill.mat")

X = mill_data["X"]  # sensor data
VB = mill_data["VB"]  # tool wear values

# Example loading QIT dataset
qit_data = pd.read_csv("qit_dataset.csv")
```

Description: Loads source (UCI Milling) and target (QIT titanium) datasets for cross-material analysis.

A.2 Data Preprocessing and Windowing

```
WINDOW_SIZE = 512

def create_windows(X, y, window_size):
    X_windows, y_windows = [], []
    for i in range(0, len(X) - window_size):
        X_windows.append(X[i:i+window_size])
        y_windows.append(y[i+window_size])
    return np.array(X_windows), np.array(y_windows)
```

Description: Converts continuous sensor signals into fixed-length time-series windows for model input.

A.3 Data Normalization

```
def normalize_window(window):
    mean = np.mean(window, axis=0)
    std = np.std(window, axis=0) + 1e-8
    return (window - mean) / std
```

Description: Applies per-window z-score normalization to stabilise model training.

A.4 Dataset Splitting

```
X_train, X_val, y_train, y_val = train_test_split(
    X_windows, y_windows, test_size=0.2, random_state=42
)
```

Description: Splits data into training and validation sets for model evaluation.

A.5 CNN Feature Extractor

```
class CNNFeatureExtractor(nn.Module):
    def __init__(self):
        super().__init__()
        self.conv = nn.Sequential(
            nn.Conv1d(3, 32, kernel_size=5),
            nn.ReLU(),
            nn.Conv1d(32, 64, kernel_size=5),
            nn.ReLU()
        )

    def forward(self, x):
        return self.conv(x)
```

Description: Extracts spatial features from multi-channel sensor signals using convolutional layers.

A.6 LSTM Temporal Model

```
class LSTMModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.lstm = nn.LSTM(input_size=64, hidden_size=128, batch_first=True)

    def forward(self, x):
        output, _ = self.lstm(x)
        return output[:, -1, :]
```

Description: Captures temporal dependencies in sequential machining signals.

A.7 Domain-Invariant Feature Fusion (MAFF Module)

```
class DomainInvariantFusion(nn.Module):
    def __init__(self, channels):
        super().__init__()
        self.attention = nn.Sequential(
            nn.Linear(channels, channels),
            nn.Sigmoid()
        )

    def forward(self, x):
        weights = self.attention(x.mean(dim=2))
        return x * weights.unsqueeze(2)
```

Description: Applies channel-wise attention to extract material-invariant features.

A.8 Full Model Architecture (MAFF Model)

```
class MAFFModel(nn.Module):
    def __init__(self):
        super().__init__()
        self.cnn = CNNFeatureExtractor()
        self.fusion = DomainInvariantFusion(64)
        self.lstm = nn.LSTM(64, 128, batch_first=True)
        self.fc = nn.Linear(128, 1)

    def forward(self, x):
        x = self.cnn(x)
        x = self.fusion(x)
        x = x.permute(0, 2, 1)
        x, _ = self.lstm(x)
        return self.fc(x[:, -1, :])
```

Description: Combines CNN, fusion layer, and LSTM to predict tool wear.

A.9 Model Training Loop

```
optimizer = torch.optim.Adam(model.parameters(), lr=5e-4)
criterion = nn.MSELoss()

for epoch in range(epochs):
    for X_batch, y_batch in train_loader:
        optimizer.zero_grad()
        outputs = model(X_batch)
        loss = criterion(outputs.squeeze(), y_batch)
        loss.backward()
        optimizer.step()
```

Description: Trains the model using Adam optimiser and mean squared error loss.

A.10 Learning Rate Scheduler

```
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(
    optimizer, mode='min', patience=5
)
```

Description: Dynamically reduces learning rate based on validation loss.

A.11 Model Evaluation

```
def evaluate(model, X_test, y_test):
    preds = model(X_test).detach().numpy()
    r2 = r2_score(y_test, preds)
    mae = mean_absolute_error(y_test, preds)
    return r2, mae
```

Description: Computes performance metrics such as R^2 and MAE.

A.12 Zero-Shot Cross-Domain Testing

```
r2_zero, mae_zero = evaluate(model, X_qit, y_qit)
```

Description: Evaluates model trained on steel directly on titanium dataset.

A.13 Few-Shot Adaptation

```
model_finetune = train(model, X_qit_small, y_qit_small)
```

Description: Fine-tunes model using a small subset of target-domain data.

A.14 Signal Ablation Study

```
X_force_only = X[:, :, 0:1]
X_vibration_only = X[:, :, 1:2]
```

Description: Evaluates model performance using individual sensor channels.

A.15 Latency Testing

```
start = time.time()
_ = model(X_sample)
latency = time.time() - start
```

Description: Measures inference time for real-time deployment feasibility.

A.16 Statistical Validation

```
stat, p_value = wilcoxon(maff_errors, baseline_errors)
```

Description: Performs statistical test to compare model performance.

APPENDIX B- GitHub Repository Link

The complete implementation of this project, including dataset preprocessing, model training, and evaluation scripts, is available at:

GitHub Repository: https://github.com/Manojkumargunupudi36/Material-Agnostic-Feature-Fusion-for-Tool-Condition-Monitoring

The repository includes:

- Data preprocessing scripts

- Model architecture implementation

- Training and evaluation pipeline

- Results reproduction instructions

Appendix C-LOGBOOK TABLE

Table: Supervisor Meeting Log

Date	Time	Topic Discussed	Supervisor Feedback	Action Taken
13-02-2026	13:00–14:00	Project title discussion	Refine title and clearly define research focus	Updated title and aligned objectives with research aim
22-02-2026	14:00–15:00	Chapter 1 – Introduction	Improve problem statement and add supporting references	Revised introduction and strengthened problem statement
27-02-2026	16:00–17:00	Literature Review (initial draft)	Improve structure and include critical analysis	Reorganised sections and added analytical content
06-03-2026	16:00–17:00	Chapter 1 & Literature Review	Clearly link literature to research gap	Added research gap and improved logical flow
13-03-2026	16:00–17:00	Literature Review (refinement)	Include research gap in each subsection	Updated all subsections with specific research gaps
27-03-2026	15:00–16:00	Methodology & Analysis	Improve dataset explanation and clarity of methodology	Revised Chapter 3 and added dataset tables

Feedback was applied at each stage, which improved the overall quality and structure of the report.

Appendix D – Ethics Approval

Certificate of Ethical Approval

Applicant:	Manoj Gunupudi
Project Title:	Material-Agnostic Feature Fusion for Tool Condition Monitoring: Bridging the Generalisation Gap Between Steel and Titanium Alloys

This is to certify that the above named applicant has completed the Coventry University Ethical Approval process and their project has been confirmed and approved as Low Risk

Date of approval:	22 Feb 2026
Project Reference Number:	P192917